

IIC2433 - Minería de Datos

🕒 Creado	@19 de marzo de 2025 15:16
🏷️ Etiquetas	

Índice

[Índice](#)

[Data Mining](#)

[Procesamiento de Datos](#)

[PCA \(Principal Component Analysis\)](#)

[Proceso PCA](#)

[Distancias y Similitud](#)

[T-SNE](#)

[Stochastic Neighbor Embedding \(SNE\)](#)

[Divergencia KL \(Kullback-Leibler\)](#)

[Model Complexity](#)

[t-SNE](#)

[Uniform Manifold Approximation and Projection \(UMAP\)](#)

[Diferencia con t-SNE](#)

[Local Outlier Factor \(LOF\)](#)

[LOF vs IQR](#)

[Machine Learning](#)

[Regresión Lineal](#)

[Ordinary Least Squares](#)

[Regresión Polinómica](#)

[Regresión Logística](#)

[Vecinos Cercanos \(KNN\)](#)

[Pairwise distance](#)

[KD-Tree](#)

[Ball-Tree](#)

[Métricas de Clasificación](#)

[Cross-Validation](#)

[Árboles de Decisión](#)

[Random Forest](#)

[Out-of-Bag \(OOB\)](#)

[Ensemble Models](#)

[Naive Bayes](#)

[Aprendizaje No Supervisado](#)

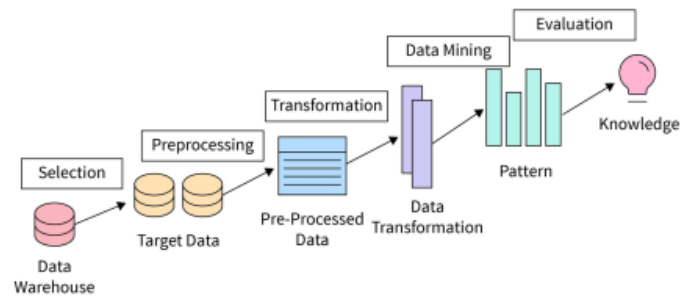
- [Clustering](#)
- [K-Means](#)
 - [Método del codo \(Elbow\)](#)
- [DBSCAN](#)
 - [Gráfico k-dist](#)
- [Clustering Jerárquico](#)
 - [Dendograma](#)
- [HDBSCAN](#)
- [Gaussian Mixture Model \(GMM\)](#)
 - [Mixture Models](#)
 - [GMM](#)
 - [Aprender Parámetros GMM](#)
 - [Expectation-Maximization \(EM\)](#)
 - [Asignación de Clusters](#)
- [Reglas de Asociación](#)
- [Redes Bayesianas](#)
 - [Independencia y Dependencia Condicional](#)
 - [Grafos Dirigidos Acíclicos \(DAGs\)](#)
 - [Inferencia Student Bayesian Network](#)
 - [Inferencia: Eliminación de Variables](#)
- [Texto en Minería de Datos](#)
 - [Conceptos Importantes](#)
 - [Conceptos de NLP](#)
- [Representación de Texto](#)
- [Bag of Words \(BoW\)](#)
- [TF-IDF](#)
- [Modelos de Lenguaje](#)
 - [Vector-Space Model](#)
 - [Word2Vec](#)
 - [Modelos de lenguaje](#)
 - [Embedding de Texto](#)
- [Atención](#)
- [Transformer](#)
- [Bert](#)
 - [Modelos Pre-entrenados](#)
 - [Fine-Tuning](#)

Data Mining

“La minería de datos es la extracción de información implícita, previamente desconocida y potencialmente útil de los datos. Se centra en construir programas informáticos que examinan bases de datos automáticamente en busca de regularidades o patrones”

→ Es un proceso que consta de 5 pasos:

Knowledge Discovery in Databases (KDD)

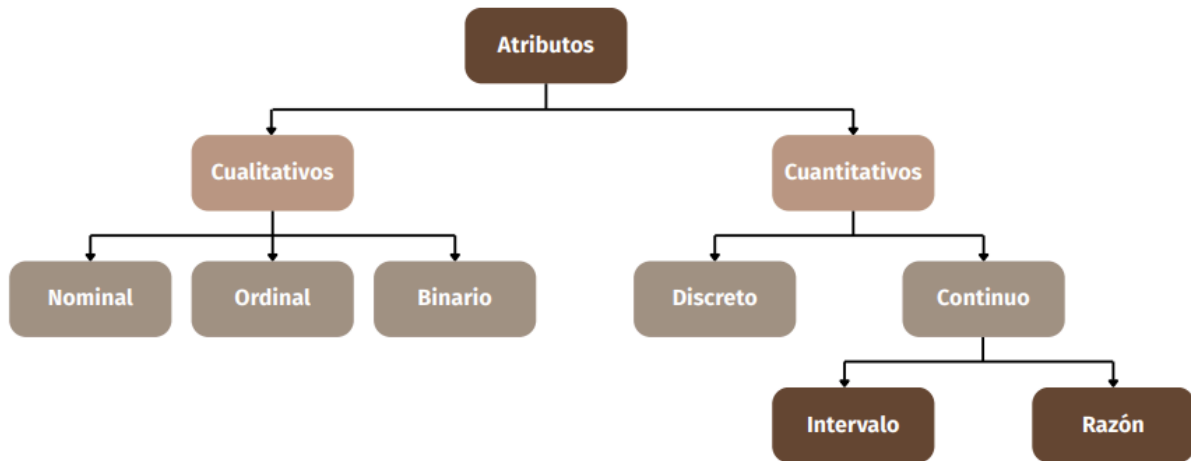


Importancia de Data Mining

1. Toma de decisiones basada en datos: Extrae patrones que respaldan decisiones estratégicas en marketing, finanzas, operaciones o innovación
2. Optimización de procesos y eficiencia: Permite detectar cuellos de botella y planificar mantenimiento predictivo
3. Ventaja competitiva sostenible: Permite descubrir oportunidades de mercado y reaccionar a nuevos comportamientos del cliente

Procesamiento de Datos

- Datos estructurados
- Datos no estructurados
- Tipos de atributos



- Data portability
 - Numérico a categórico
 - Categórico a numérico
- Data Cleaning
 - Datos faltantes (se pueden eliminar los registros que contengan un valor faltante)
 - Datos erróneos
 - Datos inconsistentes
 - Scaling (datos expresados en escalas muy distintas)

Standardization:

$$z_i^j = \frac{(x_i^j - u_j)}{\sigma_j}$$

Min-Max Scaling:

$$y_i^j = \frac{x_i^j - \min_j}{\max_j - \min_j}$$

- Data Reduction
 - Representar los datos de forma más compacta
 - Cuando los datos son más pequeños es más fácil aplicar algoritmos costosos computacionalmente
 - Pérdida de información

Sampling

- Muestreo sin remplazo de un dataset de n filas es simplemente tomar aleatoriamente un total de $n \cdot f$ filas

- En el *stratified* sampling se selecciona una **misma cantidad** de ejemplo por "clase"

Simple random sample



Stratified sample

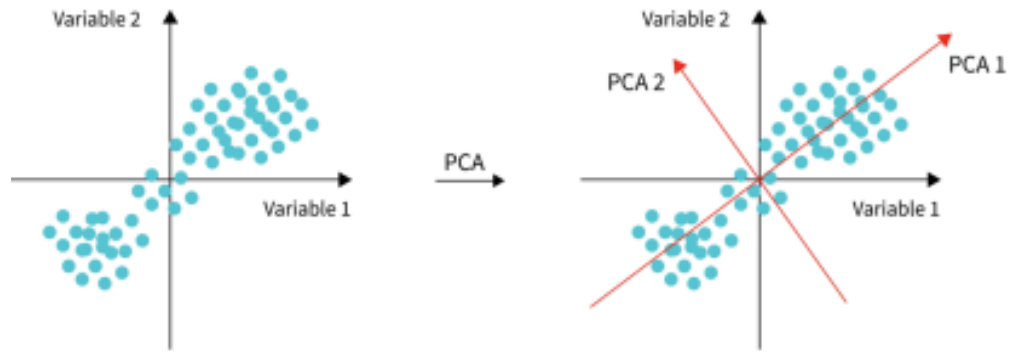


PCA (Principal Component Analysis)

Busca capturar la máxima varianza de los datos en el menor número de dimensiones posible. Reduce la cantidad de columnas que se tienen

- Simplificar el dataset para análisis, visualización o como preprocesamiento antes de aplicar otros algoritmos de Machine Learning.
- Indica la dirección donde hay mayor varianza

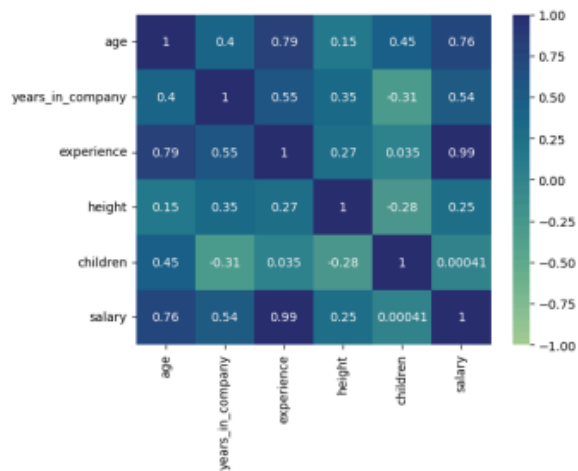
1. Reducir la dimensionalidad de los datos (menos variables).
2. Mantener la mayor cantidad posible de información (en términos de varianza).
3. Eliminar redundancia debida a correlaciones entre variables



Correlación: Pearson

$r =$

$$\frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$



Proceso PCA

Tenemos un conjunto de datos

$$D \in \mathbb{R}^{n \times d}$$

n cantidad de filas y d número de dimensiones (columnas)

1. Centrando los datos

$$\mu = (\mu_1, \mu_2, \dots, \mu_d) \quad \mu_j = \frac{1}{n} \sum_{k=1}^n x_j^{(k)}$$

→ centrar matriz con las medias: $X = D - 1\mu$

2. Matriz de covarianza

$$C = \frac{1}{n} X^T X$$

$$\begin{bmatrix} \text{Cov}(x, x) & \text{Cov}(x, y) & \text{Cov}(x, z) \\ \text{Cov}(y, x) & \text{Cov}(y, y) & \text{Cov}(y, z) \\ \text{Cov}(z, x) & \text{Cov}(z, y) & \text{Cov}(z, z) \end{bmatrix}$$

3. Descomposición en autovalores y autovectores

$$C = P \Lambda P^T$$

- $P \rightarrow$ autovectores (direcciones de los nuevos ejes)
- $\Lambda \rightarrow$ autovalores (cuánta varianza captura cada autovector)
 - Los autovalores están ordenados de mayor a menor.

4. Construcción de componentes principales

Sea la matriz $P_k = [v_1 \ v_2 \ \dots \ v_k]$,

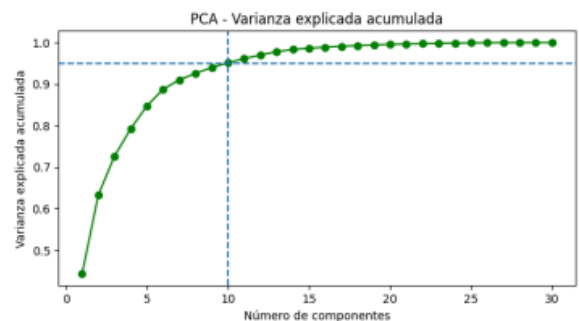
la proyección de los datos centrados sobre los primeros k componentes es

$$Z = X P_k$$

```
data = load_breast_cancer()
X = data.data # (filas, columnas)

X_std = StandardScaler().fit_transform(X)

# PCA (todas las componentes)
pca = PCA(n_components=X_std.shape[1], svd_solver='full')
X_pca = pca.fit_transform(X_std)
```



Distancias y Similitud

Dada una pareja de objetos O_1 y O_2

- Similitud: $Sim(O_1, O_2)$ (valores altos $\rightarrow$ más similares)
- Distancia: $Dist(O_1, O_2)$ (valores bajos $\rightarrow$ más similares)

Es clave para identificar objetos o patrones similares/diferentes. La elección entre función de distancia o similitud depende del dominio (datos espaciales, texto, series temporales, etc.)

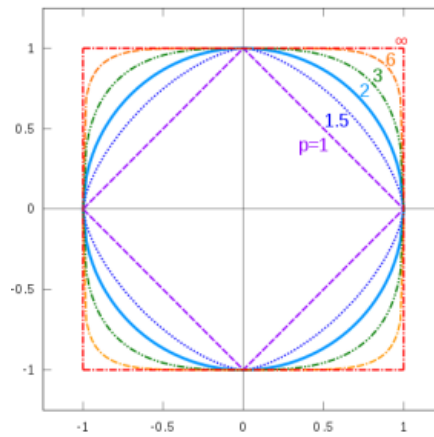
L_p -Norm (Minkowski)

La distancia más común para data cuantitativa es

L_p -Norm

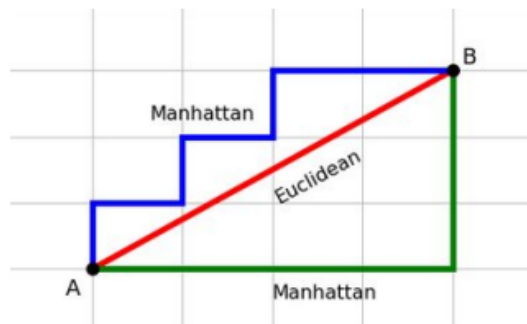
$$Dist(\bar{X}, \bar{Y}) =$$

$$\left(\sum_{i=1}^d |x_i - y_i|^p \right)^{\frac{1}{p}}$$

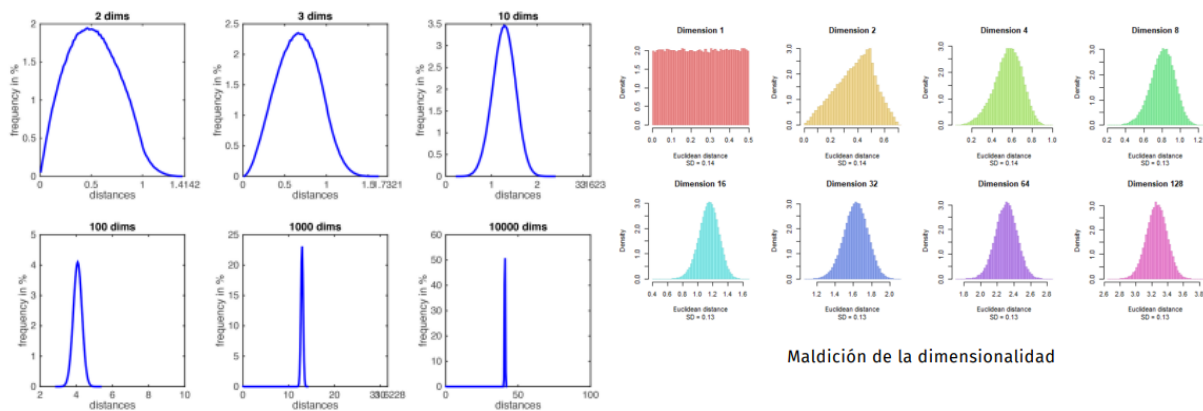


Casos especiales de L_p -Norm

- $Dist_{Manhattan}(X, Y) = \sum_{i=1}^d |x_i - y_i|$
- $Dist_{Euclidiana}(X, Y) = \sqrt{\sum_{i=1}^d |x_i - y_i|^2}$



Distancia Euclidiana



Maldición de la dimensionalidad

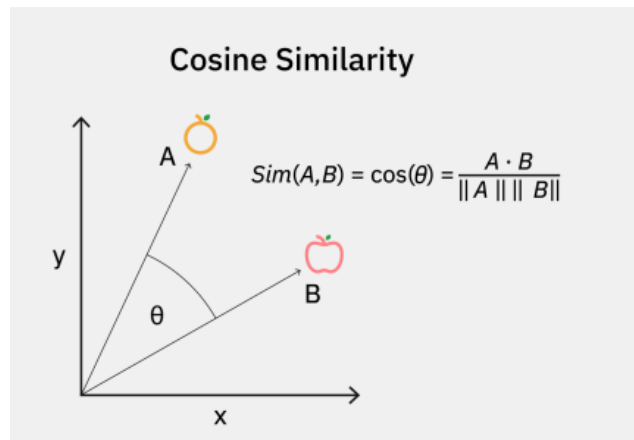
Similitud Coseno

Para dos vectores de 100 dimensiones $X = [x_0, x_1, \dots, x_{99}]$ $Y = [y_0, y_1, \dots, y_{99}]$

El producto interno se calcula como, $\langle X, Y \rangle = x_0y_0 + x_1y_1 + \dots + x_{99}y_{99}$

Geométricamente, $\langle X, Y \rangle = |X| \cdot |Y| \cdot \cos \theta$

Despejando, obtenemos $Sim_{cos}(X, Y) = \cos \theta$



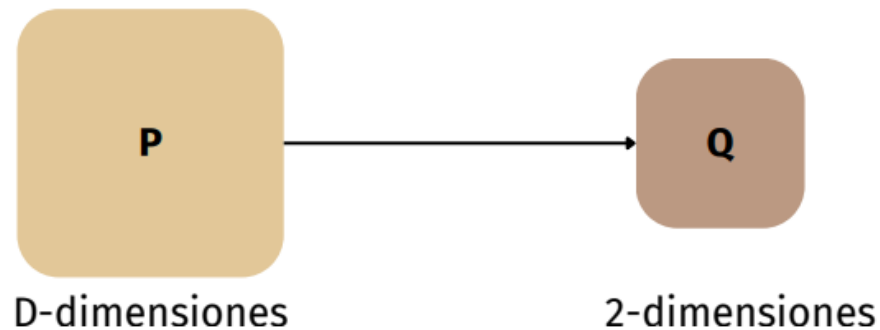
Es muy útil para trabajar con textos

T-SNE

Stochastic Neighbor Embedding (SNE)

Se busca proyectar datos en 2 o 3 dimensiones para visualización.

Dados puntos de alta dimensión $x_i \in \mathbb{R}^D (i = 1, \dots, n)$, t-SNE busca encontrar representaciones que conserven vecindades locales $y_i \in \mathbb{R}^d$

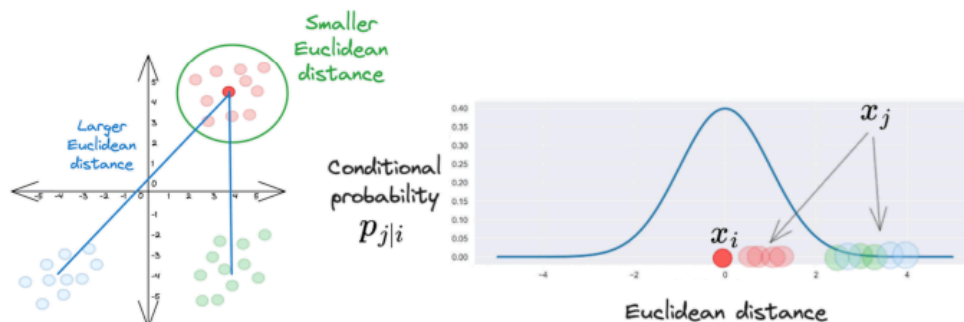


Esta visualización preserva distancias del espacio original

SNE: Similitud en alta dimensión (P)

Para cada punto, definimos una distribución condicional sobre sus vecinos:

$$p_{j|i} = \frac{\exp(-||x_i - x_j||^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-||x_i - x_k||^2 / 2\sigma_i^2)}, \quad \sigma_i \text{ es el ancho de la gaussiana del punto } i$$



SNE: Similitud en baja dimensión (Q)

Para cada punto, definimos una distribución condicional sobre sus vecinos:

$$q_{j|i} = \frac{\exp(-\|y_i - y_j\|^2)}{\sum_{k \neq i} \exp(-\|y_i - y_k\|^2)},$$

En baja dimensión no hay sigma, el embedding debe ser coherente en escala para todos los puntos → se fija un σ global

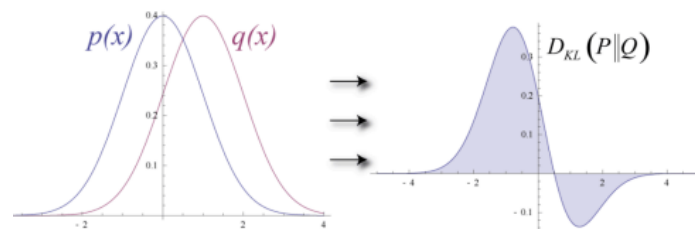
Divergencia KL (Kullback-Leibler)

Determina en qué medida una distribución de probabilidad se desvía de otra distribución de referencia

$$KL(P||Q) = \sum_{x \in X} P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$

Si el área es 0, entonces las distribuciones son iguales. A medida que va subiendo el número, más diferentes son las distribuciones.

Si son similares, significa que en alta dimensión y en baja dimensión, los datos son similares



En SNE:

Determina en qué medida una distribución de probabilidad se desvía de otra distribución de referencia

$$c = \sum_i KL(P_i||Q_i) = \sum_i \sum_{j|i} \log \left(\frac{p_{j|i}}{q_{j|i}} \right)$$

Debemos minimizar este "costo" de forma que la diferencia entre ambas distribuciones sea la menor posible:

Model Complexity

Entropía: Mide la incertidumbre promedio o la cantidad de información esperada en una distribución de probabilidad

$$H(P_i) = - \sum_j p_{j|i} \log_2 p_{j|i}$$

$$\text{Perp}(P_i) = 2^{H(P_i)}$$

```
tsne = TSNE(n_components=2, perplexity=30)
X_embedded = tsne.fit_transform(X_scaled)
```

- Se fija una perplejidad objetivo **Perp > 0** (típicamente 5-50)
- Se halla cada σ_i tal que $\text{Perp}(P_i)$ se asemeja a la perplejidad objetivo

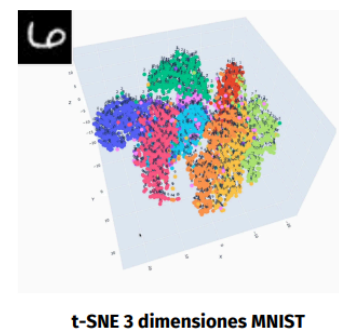
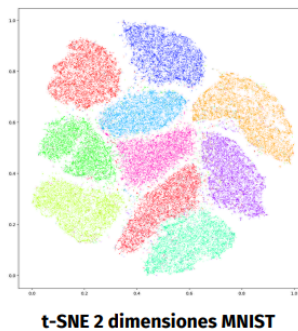
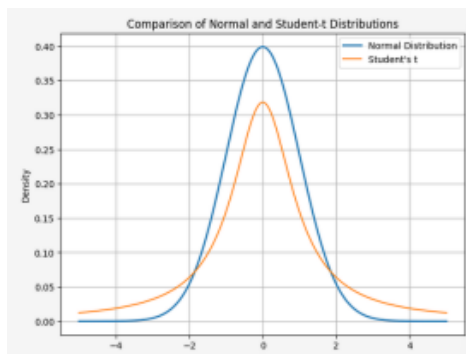
t-SNE

En baja dimensión cambia a una **t-Student** con 1 grado de libertad

$$q_{ij} = \frac{1 + ||y_i - y_j||^2)^{-1}}{\sum_{k \neq l} (1 + ||y_k - y_l||^2)^{-1}}$$

Define distribuciones simétricas

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

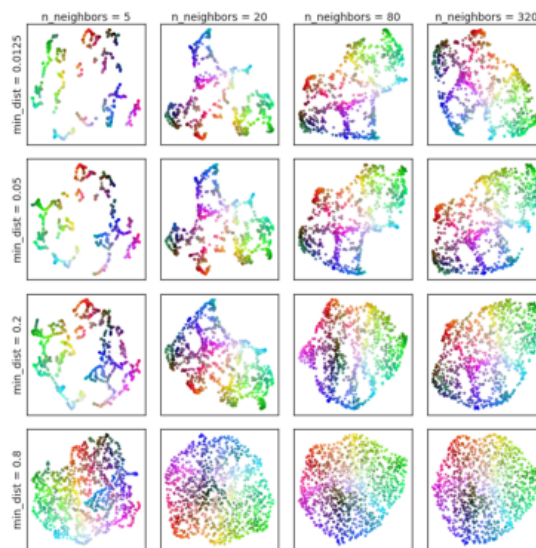


Es útil visualizar para poder sacar conclusiones, observar los clusters, y saber si tiene sentido utilizar los datos

Uniform Manifold Approximation and Projection (UMAP)

Es un algoritmo de reducción de dimensionalidad parecido a t-SNE, pero con una teoría más sólida por debajo:

- Supuesto de variedad: los datos de alta dimensión X viven en una variedad (manifold) de dimensión mucho más baja dentro de $\mathbb{R}^D$.
- Aproximación fuzzy-topológica: la estructura de vecinos se representa con un grafo difuso (fuzzy graph) que codifica las relaciones de cercanía entre puntos.

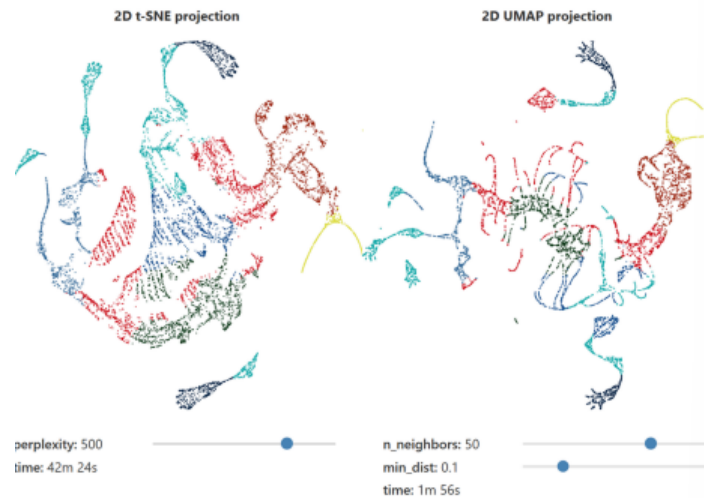


Un grafo difuso entrega la probabilidad de que exista una conexión entre nodos

Diferencia con t-SNE

1. Modelo teórico: Se basa en geometría de variedades y topología algebraica
2. Estructura global: Preserva mejor la forma global de los datos.
3. Parámetros clave:
 - n_neighbors: vecinos considerados para aproximar métrica local

- min_dist: separación entre puntos cercanos.

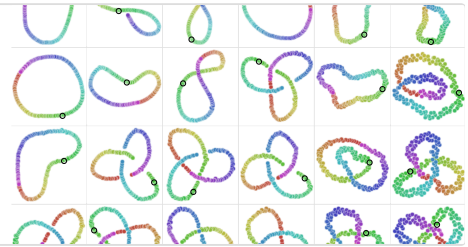


Una ventaja que tiene UMAP sobre t-SNE es el tiempo que demora en correr. Cuando se tiene una database excesivamente grande, y se quiere graficar todos los datos, conviene usar UMAP, ya que éste será capaz de dar la respuesta más rápido

Understanding UMAP

UMAP is a new dimensionality reduction technique that offers increased speed and better preservation of global structure.

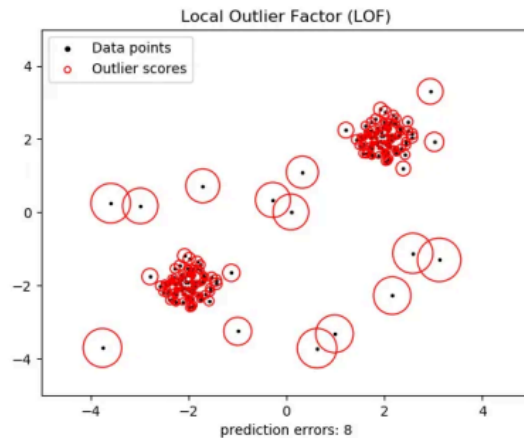
<https://pair-code.github.io/understanding-umap/>



Local Outlier Factor (LOF)

El método LOF detecta outliers (valores atípicos) basado en densidad. Comparar la densidad local de un punto con la densidad de sus vecinos más cercanos:

- Si un punto está en una región mucho menos densa que la de sus vecinos, se considera un outlier.
- Si la densidad es similar a la de sus vecinos, se considera un punto normal



LOF vs IQR

El método IQR de la primera clase es un método global y **unidimensional**. Mientras que LOF es **multidimensional** y basado en **densidad local**

- IQR
 - Los datos son univariados (solo una variable)
 - Método rápido, simple y robusto
 - Los outliers son valores muy lejanos al rango central
- LOF
 - Datos son multivariados
 - Detectar outliers locales

Machine Learning

La minería de datos y el aprendizaje de máquina se traslapan y no tienen límites claros

Programación Explícita

- El programador escribe reglas explícitas para que la máquina resuelva un problema. Es decir, se le dice exactamente qué hacer paso a paso
- No "aprende" más allá de lo que se le pide

Aprendizaje de Máquinas

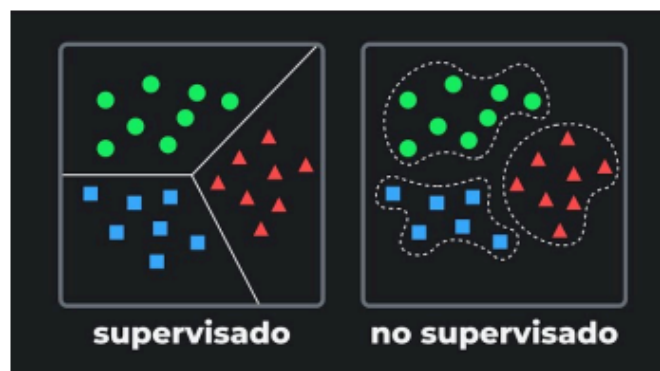
- En lugar de darle reglas, se le entrega ejemplos (datos) y un algoritmo que aprende patrones a partir de ellos.
- La máquina crea su propio modelo de reglas internas.
- El modelo aprende automáticamente qué palabras, estructuras o patrones indican spam, incluso cosas que un humano no pensó

Aprendizaje Supervisado

1. Clasificación: predecir **clases** o **etiquetas**
 - a. ejemplo: detectar si un correo es spam o no spam.
2. Regresión: predecir **valores numéricos**
 - a. ejemplo: estimar el precio de una casa.

Aprendizaje No Supervisado

1. Clustering: descubrir **grupos ocultos** en los datos.
 - a. ejemplo: segmentar clientes según hábitos de compra



Regresión Lineal

Técnica estadística donde se trata de **ajustar parámetros de una función lineal** sobre un

conjunto de datos. Se busca **predecir el valor de una variable dependiente** cuantitativa (predicha) utilizando variables independientes (predictores). Finalmente, queremos determinar cómo afecta nuestra variable independiente a la dependiente

$$Y = \alpha + \beta X$$

→ predice valores numéricos continuos

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value
0	-114.31	34.19	15.0	5612.0	1283.0	1015.0	472.0	1.4936	66900.0
1	-114.47	34.40	19.0	7650.0	1901.0	1129.0	463.0	1.8200	80100.0
2	-114.56	33.69	17.0	720.0	174.0	333.0	117.0	1.6509	85700.0
3	-114.57	33.64	14.0	1501.0	337.0	515.0	226.0	3.1917	73400.0
4	-114.57	33.57	20.0	1454.0	326.0	624.0	262.0	1.9250	65500.0
...	...	...	...	...	...	...	...	...	...
16995	-124.26	40.58	52.0	2217.0	394.0	907.0	369.0	2.3571	111400.0
16996	-124.27	40.69	36.0	2349.0	528.0	1194.0	465.0	2.5179	79000.0
16997	-124.30	41.84	17.0	2677.0	531.0	1244.0	456.0	3.0313	103600.0
16998	-124.30	41.80	19.0	2672.0	552.0	1298.0	478.0	1.9797	85800.0
16999	-124.35	40.54	52.0	1820.0	300.0	806.0	270.0	3.0147	94600.0

Dada una tabla con un conjunto de atributos numéricos $x_1, x_2, \dots, x_n$, se busca predecir un atributo numérico y . Asumimos que esta tabla representa una función $f : \mathbb{R}^n \rightarrow \mathbb{R}$ y que dicha función, es una función lineal, es decir,

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

- Relación lineal
 - La relación entre la variable dependiente y las independientes debe ser aproximación lineal.
- Independencia de los errores
 - Los errores (residuos) deben ser independientes entre sí
- Homocedasticidad (varianza constante)
 - La dispersión de los errores debe ser aproximadamente la misma a lo largo de todos los valores de X
- Normalidad de los errores
 - Los residuos deben seguir aproximadamente una distribución normal.
- No multicolinealidad (en regresión múltiple)
 - Cuando hay varias variables $x_1, x_2, \dots, x_n$ estas no deben estar fuertemente correlacionadas entre sí.

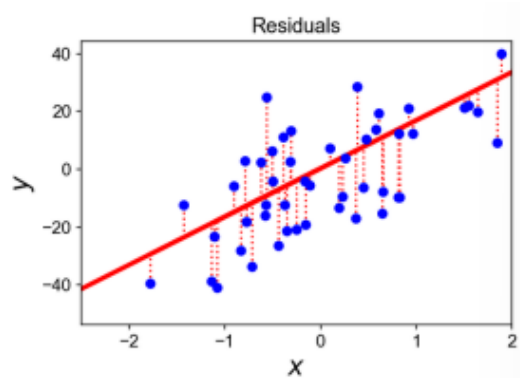
Ordinary Least Squares

1. La recta no pasará exactamente por todos los puntos $e_i = y_i - \hat{y} = y_i - (\beta_0 + \beta_1 x_i)$

2. El método de mínimos cuadrados busca la recta que minimiza la suma de los

errores al cuadrado $SSE = \min \sum_{i=1}^n e_i^2 = \min \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$

- $S(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$



3. Minimizando vamos a llegar a

- $\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$
- $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$

Derivada respecto a β_0

$$\frac{\partial S}{\partial \beta_0} = -2 \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) = 0$$

$$\sum y_i = n\beta_0 + \beta_1 \sum x_i$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

Derivada respecto a β_1

$$\frac{\partial S}{\partial \beta_1} = -2 \sum_{i=1}^n x_i (y_i - \beta_0 - \beta_1 x_i) = 0$$

$$\sum x_i y_i = \beta_0 \sum x_i + \beta_1 \sum x_i^2$$

Sustituimos $\beta_0 = \bar{y} - \beta_1 \bar{x}$

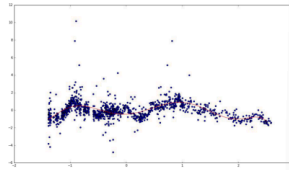
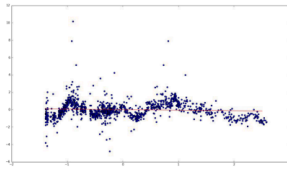
$$\sum x_i y_i = (\bar{y} - \beta_1 \bar{x}) \sum x_i + \beta_1 \sum x_i^2$$

$$\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

Lo anterior se puede resumir en: $\hat{\beta} = (X^T X)^{-1} X^T y$

donde $\hat{\beta} = (\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_d)$ — $y = (y_1, y_2, \dots, y_n)$

Regresión Polinómica



$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots + \beta_d x^d + \epsilon$$

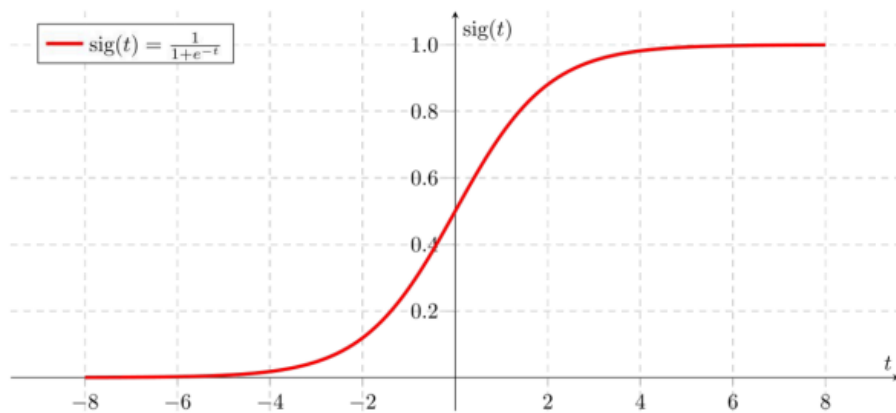
$$X_{poly} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^d \\ 1 & x_2 & x_2^2 & \dots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^d \end{bmatrix}$$

$$\hat{\beta} = (X_{poly}^\top X_{poly})^{-1} X_{poly}^\top y$$

Regresión Logística

- Está pensada para **variables categóricas binarias**

No podemos usar una recta cualquiera porque podría dar valores fuera de $[0, 1]$, entonces usamos una función que "apriete" los valores a ese rango



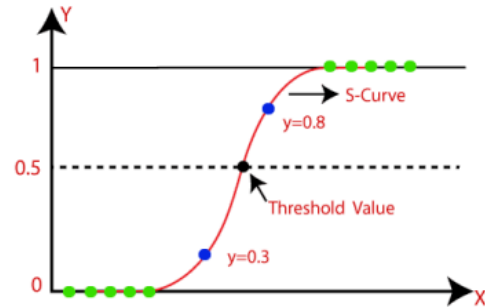
La probabilidad de que $y = 1$ dado x se modela como

$$P(y = 1|x) = \sigma(\beta_0 + \beta_1 x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

→ con varias variables, $e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p)}$

- Se usa **máxima verosimilitud**: $L(\beta) = \prod_{i=1}^n P(y_i|x_i)$
 - en su forma logarítmica: $l(\beta) = \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]$

$$\hat{y} = \begin{cases} 1, & \text{si } P(y = 1 | \mathbf{x}) \geq 0.5, \\ 0, & \text{si } P(y = 1 | \mathbf{x}) < 0.5. \end{cases}$$

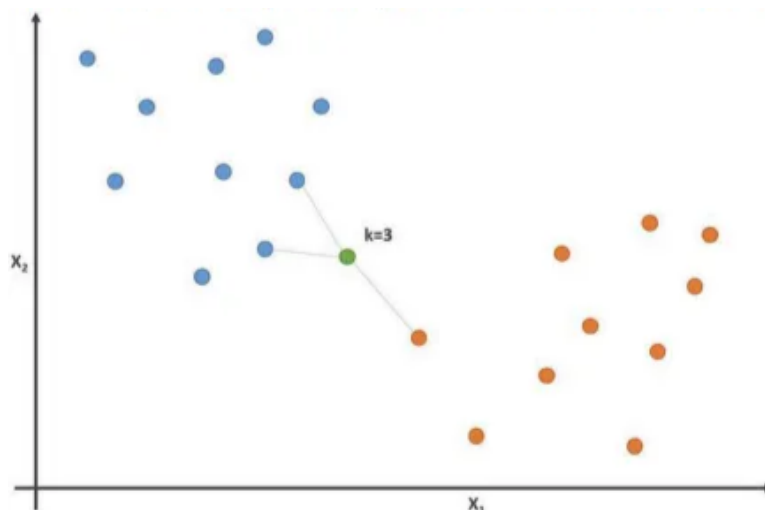


Vecinos Cercanos (KNN)

Los métodos de vecinos cercanos son la base de muchos otros métodos de aprendizaje. Consiste en **encontrar un número predefinido de muestras que están más próximas en distancia a un nuevo punto**.

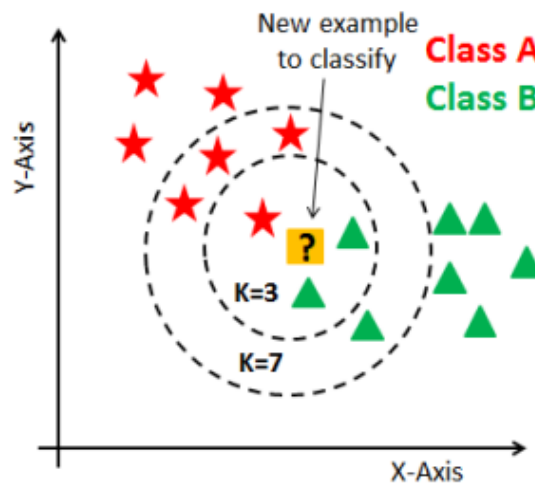
El número de muestras puede ser:

- Una constante definida por el usuario (k-vecinos más cercanos)
- Variar según la densidad local de puntos (aprendizaje basado en un radio determinado).
 - Por ejemplo, buscar 3 – NN , entrega lo siguiente



Para disponer de una estructura de vecinos cercana que podamos consultar, se usa alguno de los siguientes tres algoritmos:

- **Pairwise metric** (fuerza bruta, pocos datos)
- **BallTree** (alta dimensionalidad)
- **kd-trees** (baja dimensionalidad)

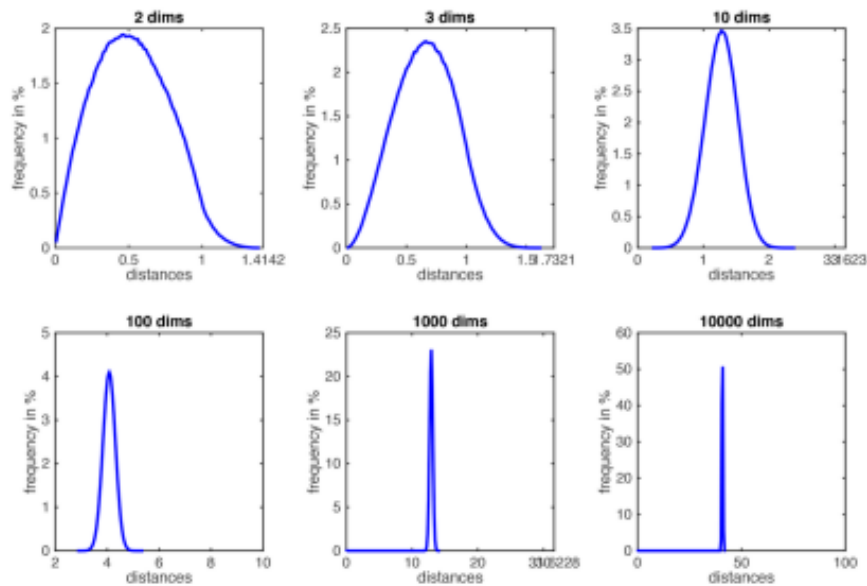


Pairwise distance

La cantidad de puntos es una de las grandes limitaciones de K-NN. Cuando hay muchos datos o muchas dimensiones, calcular la distancia euclídea contra todos los puntos se vuelve muy costoso.

Recordar que en espacios de alta dimensión, la distancia euclídea pierde sentido porque todos los puntos tienden a estar casi igual de lejos.

- Funciona para todas las distancias, pero, por defecto, se usa la eucladiana



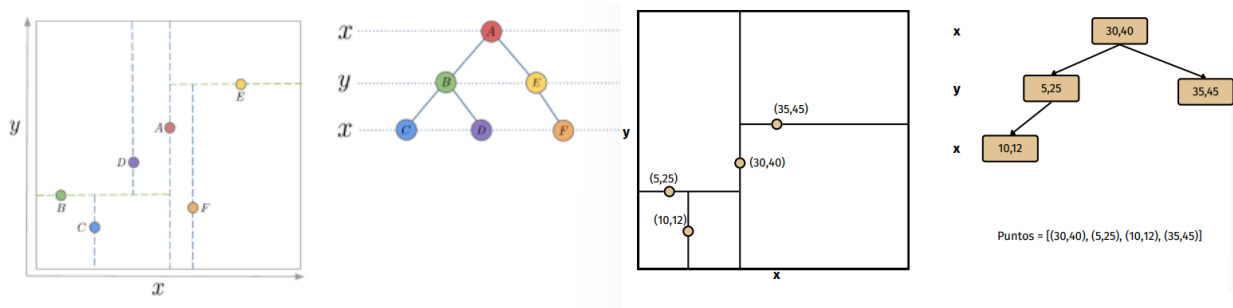
KD-Tree

Es una estructura de datos diseñada para organizar puntos en un espacio de **varias dimensiones** y así acelerar la búsqueda de vecinos cercanos.

Es un árbol binario que en cada nodo interno:

1. Se escoge una dimensión (x, y, z, ...).
2. Se selecciona un punto pivote.
3. Se divide el espacio en dos:
 - a. Subárbol izq: puntos con coordenada menor al pivote en esa dimensión.
 - b. Subárbol der: puntos con coordenada mayor.

Esto se repite de manera recursiva alternando las dimensiones (ejemplo: primero x, luego y, etc)



Siempre se divide usando la mediana del sub-espacio

Puntos = {(30,40), (5,25), (10,12), (35,45), (50,30), (70,70)}

- Eje x: Mediana {5, 10, 30, 35, 50, 70} → (30, 40)
- Eje y: Mediana {12, 25, 40} → (5, 25)
- Eje x: Mediana {5, 10, 30} → (10, 12)
- Eje y: Mediana {30, 45, 70} → (35, 45)

Clasificación

Cuando se quiere clasificar o predecir un valor para un nuevo punto:

1. Se consideran los datos de entrenamiento (ya etiquetados).
2. Para el punto nuevo, se calcula la distancia a todos los puntos de entrenamiento (euclídean, Manhattan, coseno, etc.).
3. Seleccionar los k más cercanos.
4. Decidir según esos vecinos:
 - a. Clasificación: **votación por mayoría** → la clase más repetida entre los k vecinos



```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix

# 1) Dataset
X, y = make_moons(n_samples=1500, noise=0.35, random_state=42)

# 2) Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, stratify=y)

# 3) Estandarización
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

# 5) Entrenar k=3
knn = KNeighborsClassifier(n_neighbors=3, metric="euclidean")
knn.fit(X_train_std, y_train)

# 6) Evaluación
y_pred = knn.predict(X_test_std)
y_proba = knn.predict_proba(X_test_std)[:,1]

print("Reporte de clasificación:")
print(classification_report(y_test, y_pred, digits=2))

print("Matriz de confusión:")
print(confusion_matrix(y_test, y_pred))

```

Reporte de clasificación:				
	precision	recall	f1-score	support
0	0.86	0.87	0.86	150
1	0.86	0.85	0.86	150
accuracy			0.86	300
macro avg	0.86	0.86	0.86	300
weighted avg	0.86	0.86	0.86	300
Matriz de confusión:				
[[130 20]				
[22 128]]				

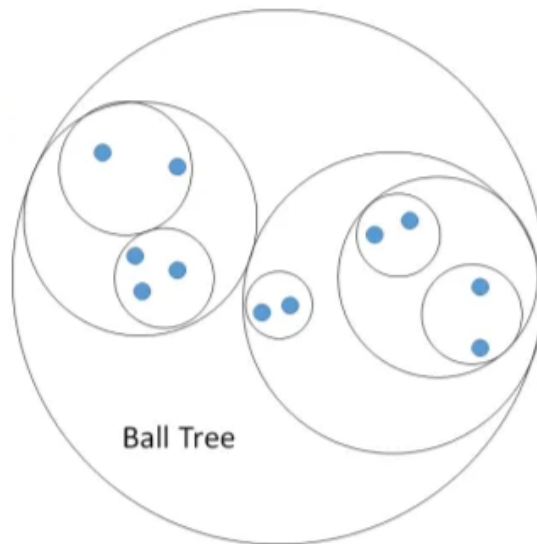
Ball-Tree

Un Ball-Tree es otra estructura de datos para vecinos cercanos, pensada para superar algunas limitaciones de los KD-tree.

Se crea un árbol binario. **Cada nodo define la esfera más pequeña** que contiene los puntos de su subárbol.

El criterio de construcción da lugar a un invariante que usaremos en búsqueda:

- Dado un punto externo t a una esfera B , su distancia a cualquier punto de B será mayor o igual que la distancia a la superficie de B



Representación 1 nodo

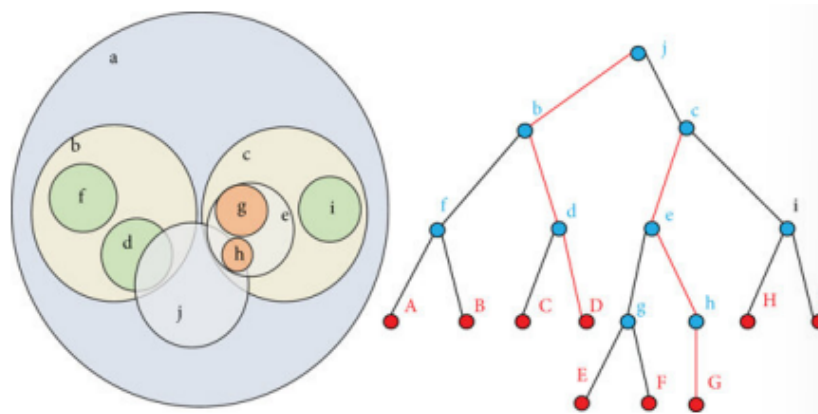
Un nodo del Ball-tree está definido por dos cosas:

$$B = \{c, r\}$$

$c \in \mathbb{R}^d$: centro del ball (centroide de los puntos que contiene).

$r \in \mathbb{R}$: radio mínimo que cubre todos los puntos:

$$r = \max \|x - c\|$$



Dado un conjunto de puntos $S \subset \mathbb{R}^d$,

$$\rightarrow c = \frac{1}{|S|} \sum_{x \in S} x, \quad r = \max \|x - c\|$$

Si $|S|$ es mayor que el tamaño mínimo de hoja (leaf size), dividir en dos subconjuntos:

- Escoger dos puntos alejados $a, b \in S$ (ej. el par más distante)
- Asignar cada punto al subconjunto más cercano:

$$S_a = \{x \in S : \|x - a\| \leq \|x - b\|\} \text{ y } S_b = \{x \in S : \|x - b\| < \|x - a\|\}$$

Búsqueda de vecino cercano

Supongamos que queremos encontrar el vecino más cercano de una consulta $q \in \mathbb{R}^d$

$$c = \frac{1}{|S|} \sum_{x \in S} x, \quad r = \max_{x \in S} \|x - c\|$$

Paso 1: Distancia mínima a un ball

$$d(q, B) = \max(0, \|q - c\| - r)$$

Si q está dentro de la esfera $\rightarrow d(q, B) = 0$

Si está afuera $\rightarrow$ es la distancia del punto al borde de la esfera.

Paso 2: Poda de subárboles

Si ya tenemos un mejor candidato con distancia d^* , y para un ball:

$$d(q, B) \geq d^*$$

podemos descartar todo el ball, porque ningún punto en él puede estar más cerca que d^*

Métricas de Clasificación

- Accuracy: mide la proporción de predicciones correctas sobre el total.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Qué tanto acerté en general
- En general, no es la mejor métrica porque, cuando hay mucha diferencia entre la cantidad de datos de lo que se quiere entrenar, el peso del accuracy se lo lleva la cantidad de datos que hay en mayor cantidad
 - Por ejemplo, si yo quiero entrenar un modelo a reconocer gatos, y le paso 1000 imágenes de perro y 10 de gatos, el modelo podría tener un 90% de accuracy, pero no achuntarle a ningún gato si no que a puros perros
- Precision: indica qué proporción de los predichos como positivos realmente lo son

$$\text{Precision} = \frac{TP}{TP + FP}$$

- de lo que predije como X, ¿cuánto realmente era X?
- Indica el porcentaje de lo que mi modelo arrojó como valor 1 (lo que estoy tratando de buscar), realmente era valor 1
- Recall: indica qué proporción de los positivos reales fueron correctamente detectados.

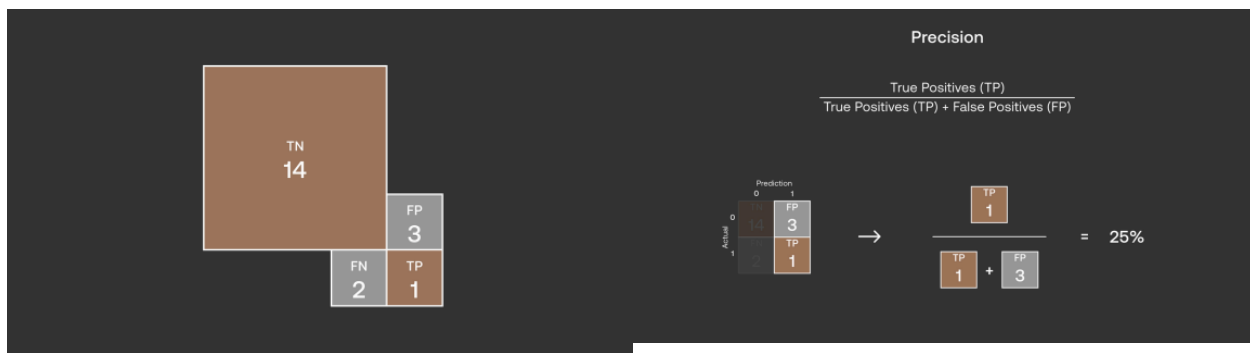
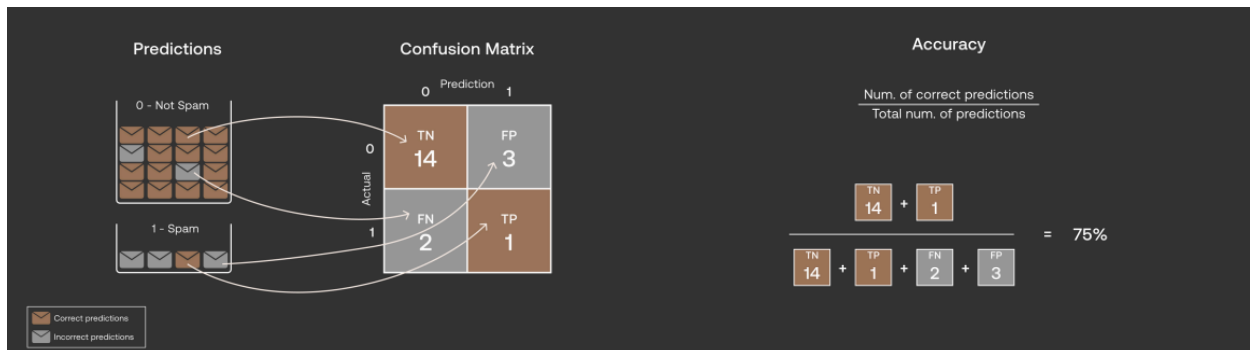
$$\text{Recall} = \frac{TP}{TP + FN}$$

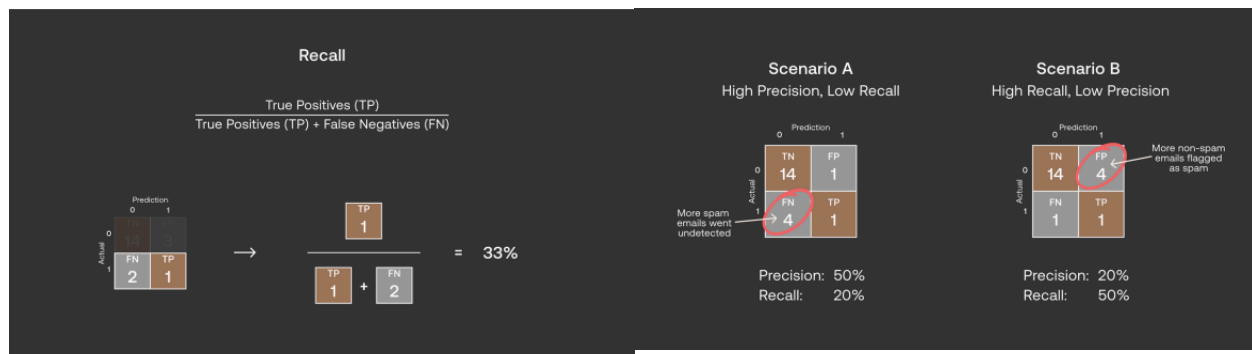
- de lo que realmente era X, ¿cuánto detecté?
- Indica el porcentaje de lo que realmente era 1, cuánto fue mi modelo capaz de identificar como 1
- F1-score: es la **media armónica** entre precisión y recall.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- equilibrio entre ambos
- Es útil para comparar
- Castiga cuando hay valores muy separados, por lo que, mientras más cercanos sean los valores, mejor

Donde **TN**: true negative, **TP**: true positive, **FN**: false negatives, **FP**: false positives



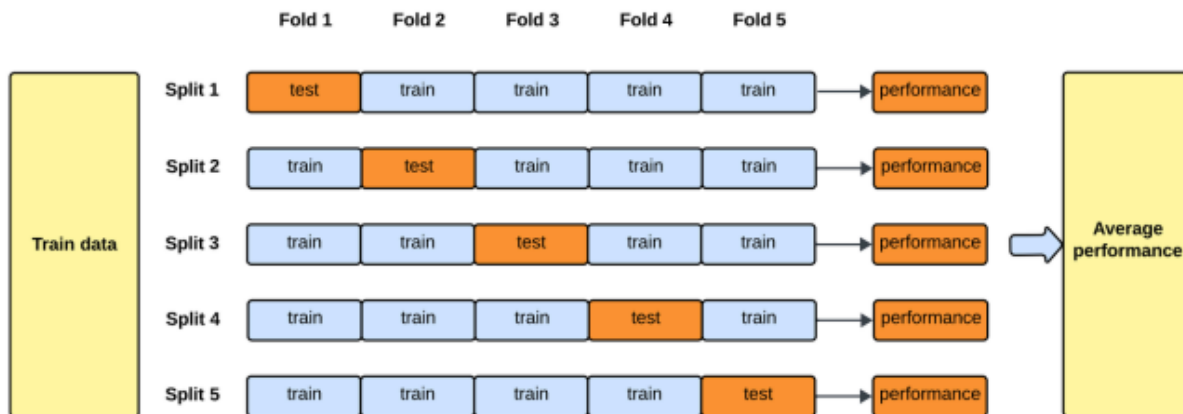


F1

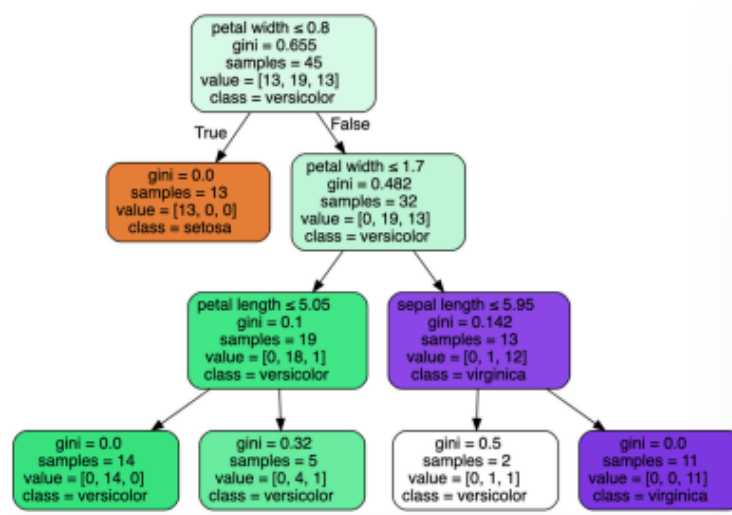
$$\frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 * 25\% * 33\%}{25\% + 33\%} = 28\%$$

Cross-Validation

Es una técnica para evaluar el desempeño real de un modelo de machine learning, reduciendo el riesgo de que los resultados dependan demasiado de la partición de los datos en train/test.



Árboles de Decisión



- Los **nodos** del árbol representan variables, las ramas representan **valores de las variables** que permiten clasificar
- Las hojas del árbol corresponden a la clasificación
- El algoritmo selecciona la característica que mejor separa los datos. Se usan *métricas de "pureza"* como
 - Gini (impureza Gini)

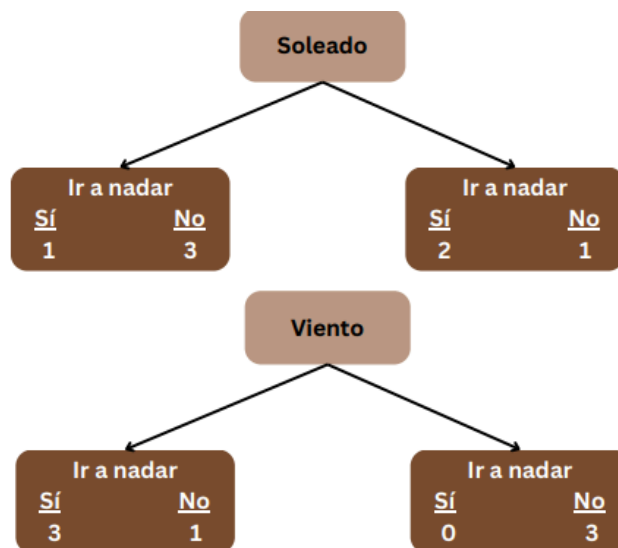
$$Gini = 1 - \sum_{i=1}^C p_i^2$$

- Más rápida de calcular (no usa logaritmos)
- Favorece la clase mayoritaria un poco más que la entropía
- Entropía (information game)

$$H = - \sum_{i=1}^C p_i \log_2(p_i)$$

- Se basa en teoría de la información
- Da una visión más *matemática* de la incertidumbre

Soleado	Viento	Edad	Ir a nadar
Sí	Leve	16	No
Sí	Fuerte	14	No
No	Leve	23	Sí
No	Leve	20	Sí
Sí	Leve	26	Sí
Sí	Fuerte	32	No
No	Fuerte	27	No



- Uno es un *nodo puro* cuando están todas las decisiones en una sola variable



La **impureza de Gini para la hoja A** es:

$$\begin{aligned} \text{Gini}_A &= 1 - (\text{prop. clase "Sí"})^2 - (\text{prop. clase "No"})^2 \\ \text{Gini}_A &= 1 - \left(\frac{1}{4}\right)^2 - \left(\frac{3}{4}\right)^2 \\ \text{Gini}_A &= 0.375 \end{aligned}$$

De forma similar para la hoja B es: $\text{Gini}_B = 0.444$

La impureza de Gini total considera una ponderación:

$$\text{Gini} = \frac{4}{7} \times 0.375 + \frac{3}{7} \times 0.444 = 0.405$$

$$\begin{aligned} H_A &= -(0.25 \log_2 0.25 + 0.75 \log_2 0.75) \\ H_A &= -(0.25 \times (-2) + 0.75 \times (-0.415)) \\ H_A &= 0.5 + 0.311 = 0.811 \end{aligned}$$

De forma similar para la hoja B es: $H_B = 0.918$

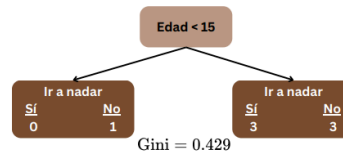
La entropía total ponderada es:

$$H_{\text{total}} = \frac{4}{7} \times 0.811 + \frac{3}{7} \times 0.918 = 0.856$$

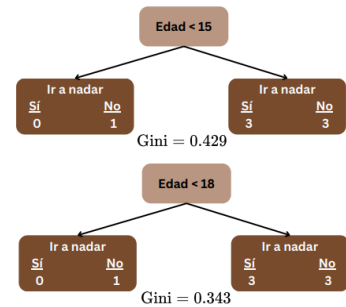
Y para datos numéricos?

Se ordenan los datos y se empiezan a por orden, en pares. Luego, se calcula el promedio y se analiza el comportamiento cuando es mayor o menor a la media encontrada

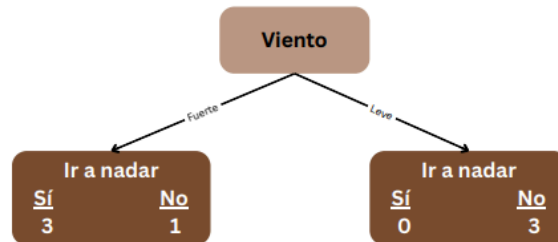
Edad	Ir a nadar
14	No
16	No
20	Sí
23	Sí
26	Sí
27	No
32	No



Edad	Ir a nadar
14	No
16	No
20	Sí
23	Sí
26	Sí
27	No
32	No



Soleado	Viento	Edad	Ir a nadar
Sí	Leve	23	No
Sí	Fuerte	14	No
No	Leve	30	Sí
No	Leve	16	Sí
Sí	Leve	27	Sí
Sí	Fuerte	18	No
No	Fuerte	20	No



Soleado: Gini = 0.405

Viento: Gini = 0.214

Edad: Gini = 0.343



Parámetros importantes:

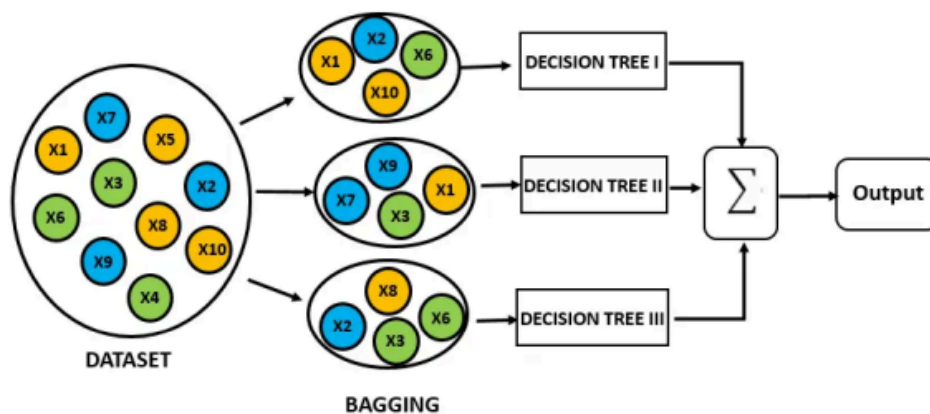
- criterion: métrica de impureza
- max_depth: límite de profundidad
- min_samples_split: mínimo de muestras para dividir un nodo
- min_samples_leaf: mínimo de muestras en una hoja

Random Forest

Un árbol de decisión por sí solo es un modelo simple y explicable, pero tiene un gran problema: tiende a sobreajustar (**overfitting**) si no se limita bien.

Leo Breiman, 2001, propuso una mejora al árbol de decisión, el cual es un modelo con muy buenos resultados y rendimiento

- Modelo basado en los árboles de decisión.
- Como su nombre lo dice, genera un bosque o selva de ellos para tomar una decisión.
- Cada árbol está formado por un subconjunto de los atributos totales.
- Para clasificar se genera una votación entre todos los árboles



Es un ensamble de muchos árboles de decisión, donde cada árbol

- Se entrena sobre un subconjunto aleatorio de los datos (**bootstrapping** → muestreo con reemplazo).
- En cada división, considera solo un subconjunto aleatorio de las variables.

Training dataset

Obs	X_1	X_2	X_3	X_4	X_5	Y
1	X_{11}	X_{21}	X_{31}	X_{41}	X_{51}	0
2	X_{12}	X_{22}	X_{32}	X_{42}	X_{52}	1
3	X_{13}	X_{23}	X_{33}	X_{43}	X_{53}	0
4	X_{14}	X_{24}	X_{34}	X_{44}	X_{54}	0
5	X_{15}	X_{25}	X_{35}	X_{45}	X_{55}	1

Bootstrap

Obs	X_1	X_3	X_4	Y
1	X_{11}	X_{31}	X_{41}	0
2	X_{12}	X_{32}	X_{42}	1
5	X_{15}	X_{35}	X_{45}	1
1	X_{11}	X_{31}	X_{41}	0
5	X_{15}	X_{35}	X_{45}	1

Obs	X_2	X_3	X_4	Y
1	X_{21}	X_{31}	X_{41}	0
3	X_{23}	X_{33}	X_{43}	0
4	X_{24}	X_{34}	X_{44}	0
3	X_{23}	X_{33}	X_{43}	0
2	X_{22}	X_{32}	X_{42}	0

Obs	X_1	X_2	X_5	Y
2	X_{12}	X_{22}	X_{52}	1
3	X_{13}	X_{23}	X_{53}	0
5	X_{15}	X_{25}	X_{55}	1
5	X_{15}	X_{25}	X_{55}	1
3	X_{13}	X_{23}	X_{53}	0

→ Se entrenan varios árboles de decisión con los distintos *bootstrapped*. Son sampleos distintos del mismo dataset

Out-of-Bag (OOB)

Es una técnica de validación interna que aparece de manera natural en los Random Forests gracias al muestreo bootstrap. Son las filas que quedan fuera de la elección

Soleado	Viento	Edad	Ir a nadar
Sí	Leve	23	No
Sí	Fuerte	14	No
No	Leve	30	Sí
No	Leve	16	Sí
Sí	Leve	27	Sí
Sí	Fuerte	18	No
No	Fuerte	20	No

Bootstrapped

Soleado	Viento	Edad	Ir a nadar
Sí	Leve	23	No
...	...	...	...
No	Fuerte	20	No

Out-of-Bag

Soleado	Viento	Edad	Ir a nadar
Sí	Leve	27	Sí
No	Fuerte	20	No

Dentro del Random Forest

- Cada árbol se entrena con una muestra bootstrap del dataset (selección aleatoria con reemplazo).
- En promedio, cada muestra bootstrap contiene alrededor del 63% de los datos originales.

¿Qué se hace con los OOB?

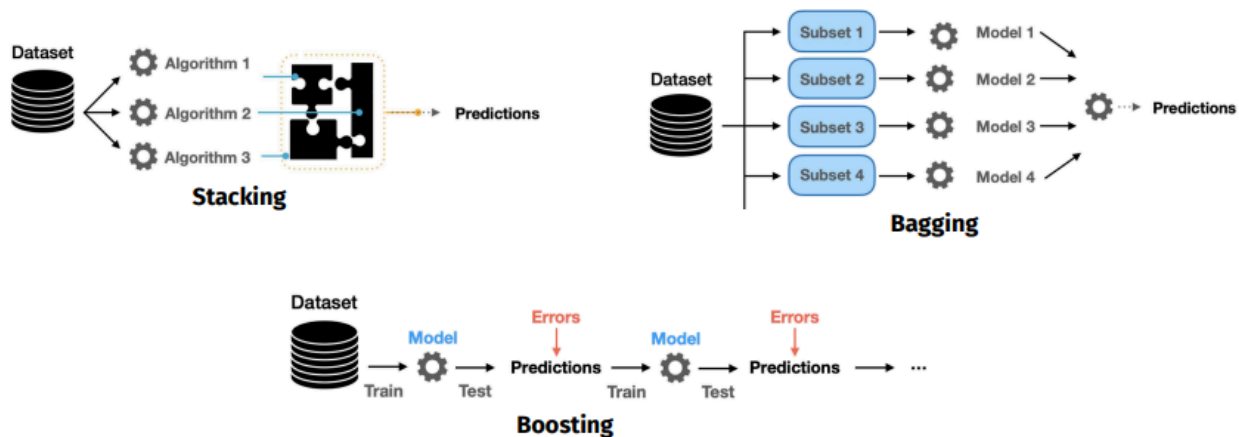
- Para cada árbol, se puede usar sus datos fuera de bolsa (OOB) para probar cómo clasifica.
- Al final, se combinan las predicciones de todos los árboles donde un ejemplo estuvo "fuera de bolsa".
- Esto genera una **estimación del error** de generalización del bosque sin necesidad de un conjunto de validación separado

! n_estimators: número de árboles → por defecto, es 100 (más árboles → más estable, pero más lento).

Ensemble Models

Es un modelo grande compuesto por varios modelos juntos/unidos

- Plantea que una mayor diversidad de modelos mejoran el performance general.
- Se pueden generar varias combinaciones, ya sea votaciones, ponderaciones o incluso, un aprendizaje continuo



Naive Bayes

Es un algoritmo de clasificación que se base en el teorema de Bayes

- La diferencia que tiene con el teorema es que se asume que las variables son **condicionalmente independientes** entre si

$$P(x|C_k) = \prod_{I=1}^n P(x_i|C_k)$$

¿Por qué solo nos interesa el numerador?

Teorema de Bayes

$$P(C_k | x) = \frac{P(x | C_k)P(C_k)}{P(x)} \quad P(C_k | x) = \frac{P(x | C_k)P(C_k)}{\sum_j P(x | C_j)P(C_j)}$$

Naïve Bayes

Cuando usamos Naive Bayes para elegir la clase más probable, nos interesa:

$$\hat{C} = \arg \max_{C_k} P(C_k | x)$$

Sustituyendo por Bayes

$$\hat{C} = \arg \max_{C_k} \frac{P(x | C_k)P(C_k)}{P(x)}$$

El denominador es el mismo para todas las clases, porque no depende de la clase. Por eso, se cancela en la comparación.

Ventajas ✓

- Muy **eficiente en entrenamiento y clasificación**, incluso con grandes volúmenes de datos.
- **Pocos datos necesarios**: funciona bien incluso con datasets pequeños.
- Adecuado para problemas de texto con miles de características (bolsa de palabras).
- A pesar de su simplicidad y la “ingenua” suposición de independencia, suele dar buenos resultados en práctica

Desventajas ●

- Suposición de independencia irrealista: **en muchos problemas reales, las variables están correlacionadas**, esto puede afectar la precisión.
- Si hay muchos atributos que no aportan, puede sesgar los resultados.
- **No capta relaciones complejas entre variables** como lo harían otros modelos más avanzados.
- Rendimiento limitado en datasets donde las dependencias entre atributos son fuertes.

Aprendizaje No Supervisado

Solamente contamos con X , los modelos buscan patrones ocultos o estructuras

Clustering

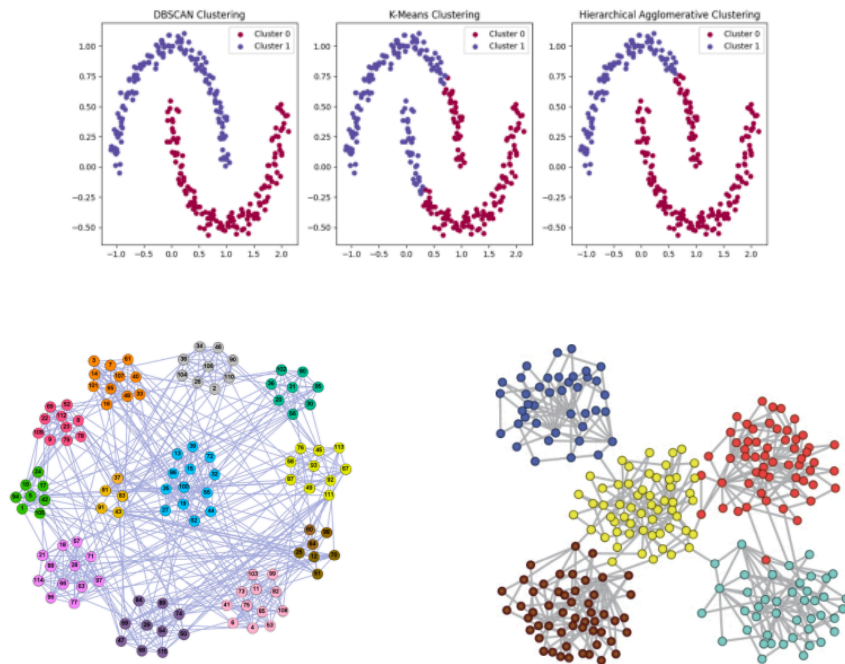
Clustering es una técnica de aprendizaje no supervisado cuyo objetivo es **agrupar un conjunto de datos en grupos** (clusters) de manera que

- Los elementos dentro de un mismo grupo sean lo más similares entre sí.

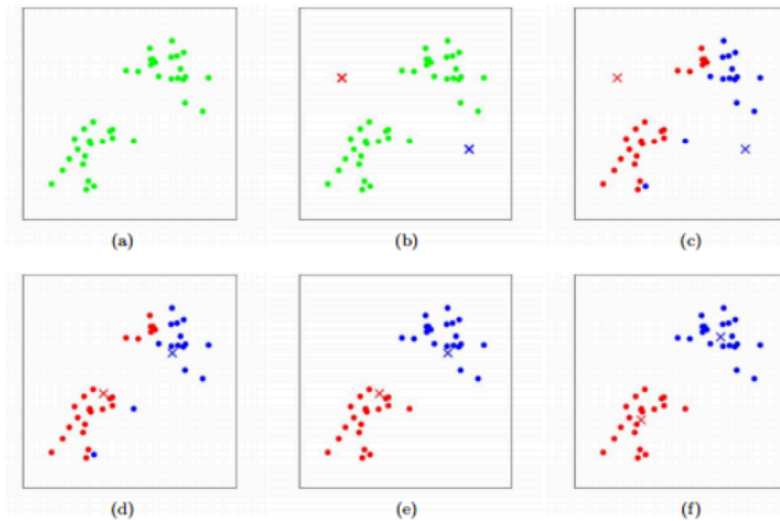
- Los elementos de grupos diferentes sean lo más distintos posible.

Aplicaciones

- Segmentación de clientes
- Análisis de imágenes
- Detección de anomalías
- Agrupación de documentos/textos.
- Biología y medicina



K-Means



(a) Tenemos nuestros datos iniciales

(b) Se eligen k puntos al azar

(c) Todos los puntos que están cerca de un punto, son rojos; y todos los puntos cerca del otro, son azul

(d) Para ambos colores, se busca el centro de masa de ese nuevo cluster

(e) Esos centros pasan a ser nuestro nuevos centroides

(f) Esto se hace hasta no tener mucha variabilidad entre los nuevos centroides

K-Means funciona como un algoritmo iterativo que divide los datos en k grupos (clusters) **según su cercanía**

1. Elegir k : decides cuántos clusters quieres.
2. Inicializar centroides: se colocan k puntos al azar (los "centros" de cada cluster).
3. Asignar puntos: cada dato se asigna al centroide más cercano (ejemplo, distancia euclidiana).
4. Actualizar centroides: calcular el **promedio de los puntos** en cada cluster y se mueve el centroide a esa posición.
5. Repetir pasos 3-4 hasta que los centroides ya no cambien (o número máximo de iteraciones).



k es la cantidad de puntos que se arrojan, por ende, la cantidad de clusters que se generan

K-Means

Problema

Tenemos el conjunto de datos:

$$X = \{x_1, x_2, \dots, x_n\}, \quad x_i \in \mathbb{R}^d$$

El objetivo es dividirlos en k clusters

$$C = \{C_1, C_2, \dots, C_k\}.$$

Función Objetivo

Minimizar las distancias entre los puntos y el centroide de su cluster:

$$J = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

Algoritmo iterativo

Inicialización: elige aleatoriamente k centroides $\mu_1, \dots, \mu_k$

Asignación: cada punto se asigna al cluster más cercano:

$$C_j = \{x_i : \|x_i - \mu_j\|^2 \leq \|x_i - \mu_l\|^2 \quad \forall l\}$$

Actualización: recalcular el centroide de cada cluster:

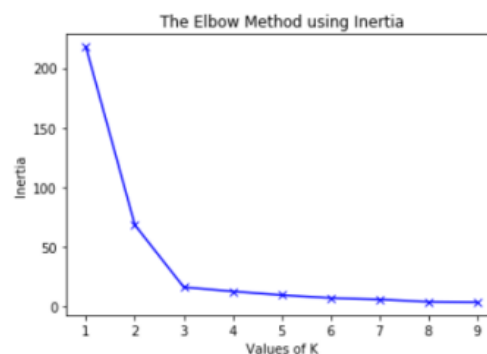
$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

Repetir hasta que los centroides no cambien.

Método del codo (Elbow)

- Calcular la función objetivo (también llamada **inercia**) para distintos valores de k .
- A medida que **augmenta k , la inercia disminuye** (más clusters = menor error).
- Se elige el k en el que la mejora empieza a ser marginal → el "codo" de la curva.

$$\text{Inertia}(k) = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

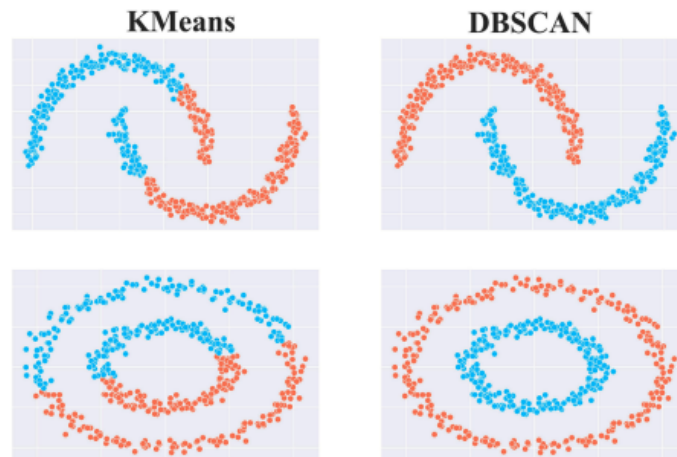


→ Donde hay un *codo*, hay un cluster

DBSCAN

Limitaciones de K-Means

- Asume que los clusters son aproximadamente esféricos y de tamaño similar
- Es **sensible a outliers**, un solo dato muy alejado puede desplazar fuertemente un centroide



DBSCAN (Density-Based Spatial Clustering of Applications with Noise) funciona de manera distinta a K-Means: en lugar de depender de centroides y un número fijo de clusters, **agrupa puntos según su densidad en el espacio**.

- Encuentra el k solo

Parámetros clave:

- **ϵ (eps)**: radio de vecindad.
- **minPts**: número mínimo de puntos necesarios en un radio ϵ para ser considerado (incluyendo si mismo).
 - Generalmente se ocupa 4 o 5 como estándar

Con esto, los puntos se clasifican como:

1. **Núcleo (core point)**: tiene al menos **minPts** vecinos dentro del radio ϵ .
2. **Borde (border point)**: está dentro de la vecindad ϵ de un punto núcleo, pero no cumple por sí mismo con **minPts**.

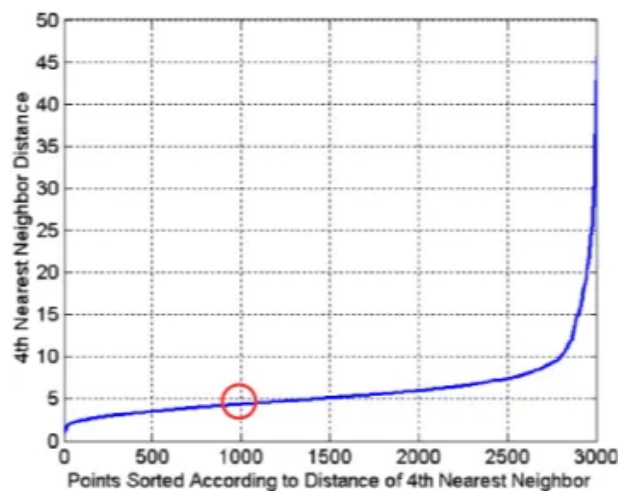
3. **Outlier**: no es núcleo ni borde (queda fuera de clusters).

Cómo funciona DBSCAN

Gráfico k-dist

El k-dist plot nos permite **definir el valor de ϵ** :

1. Para cada punto, calcular la distancia al minPts-ésimo vecino más cercano.
2. Ordenar esas distancias de menor a mayor.
3. El valor de ϵ se elige en la zona donde aparece un codo en la curva



¿Qué nos dice la imagen?

Aprox. 1000 nodos tienen distancia = 5 a su cuarto vecino (por minPts = 4)



ϵ pequeños $\rightarrow$ solo núcleos muy densos

ϵ grandes $\rightarrow$ clusters muy amplios

Clustering Jerárquico

Es un método de **agrupamiento no supervisado que busca organizar objetos en una jerarquía** de grupos o clústeres, según su similitud o distancia.

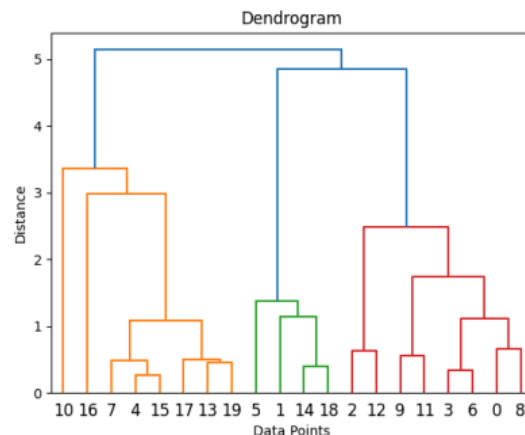
Construye una jerarquía multinivel de grupos al unir clústeres más pequeños en otros más grandes o dividir un clúster grande en otros más pequeños.

Esto da como resultado una estructura en forma de árbol conocida como **dendrograma**

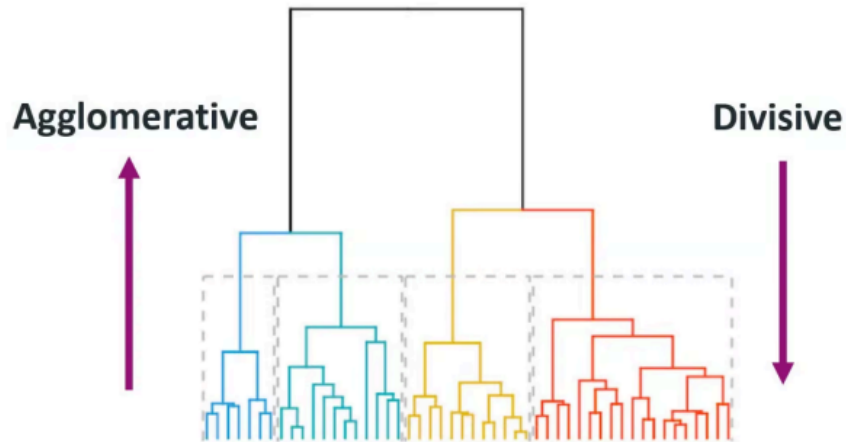
Dendrograma

En lugar de asignar cada punto directamente a un grupo (como hace K-Means), el clustering jerárquico construye un dendrograma, que muestra cómo los grupos se van formando o dividiendo paso a paso.

- La altura de las ramas en un dendrograma representa la distancia o disimilitud a la que los clústeres se unen.
- Las alturas más bajas indican clústeres que se combinan a distancias menores, lo que representa una mayor similitud.



1. **Aglomerativo (bottom-up)**: Empieza con cada punto separado y va uniendo los más similares hasta formar un clúster
2. **Divisivo (top-down)**: Empieza con todos los puntos juntos y va dividiendo el grupo en partes más pequeñas



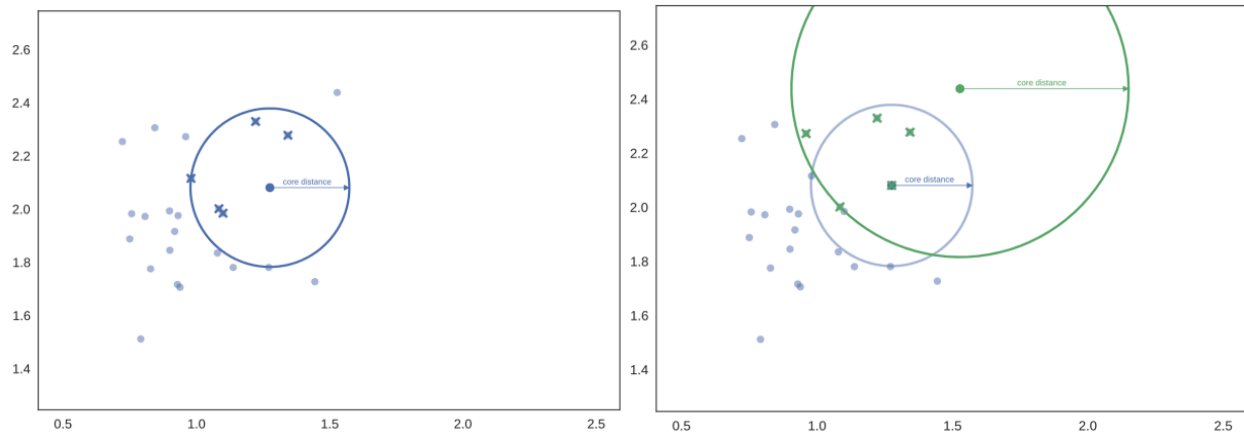
HDBSCAN

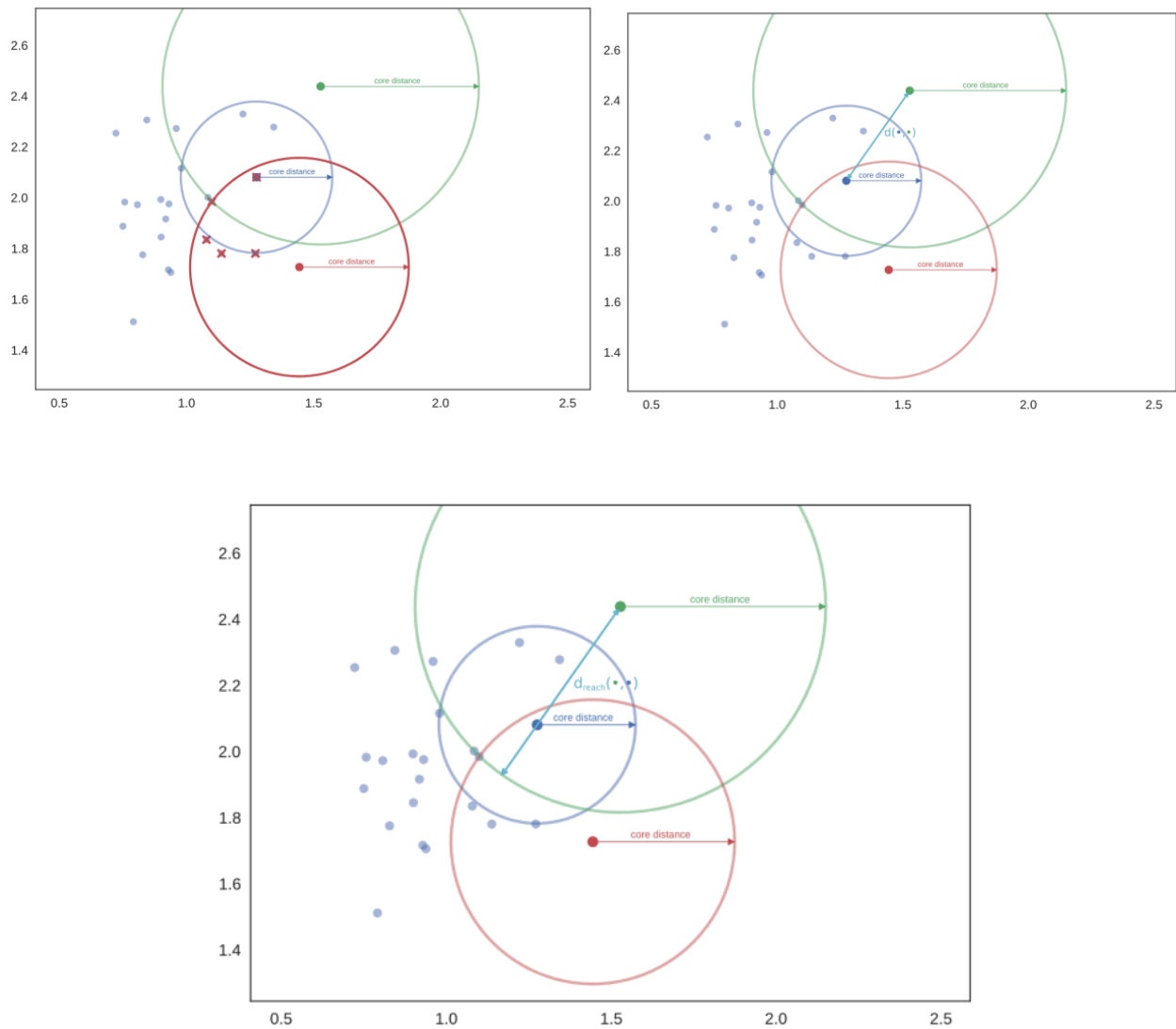
1. Mide la densidad local

Primero, estima qué tan denso es el entorno de cada punto. Para eso, calcula la distancia al vecino número k. Esa distancia se llama distancia de alcance principal (core distance): $core_k(i) = \text{distancia hasta su } k\text{-ésimo vecino mas cercano}$

2. Mutual reachability distance

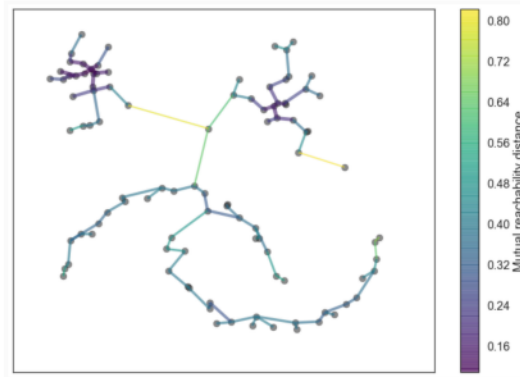
Define la distancia de alcance mutuo entre dos puntos i y j: $d_{mreach}(i, j) = \max\{core_k(i), core_k(j), dist(i, j)\}$



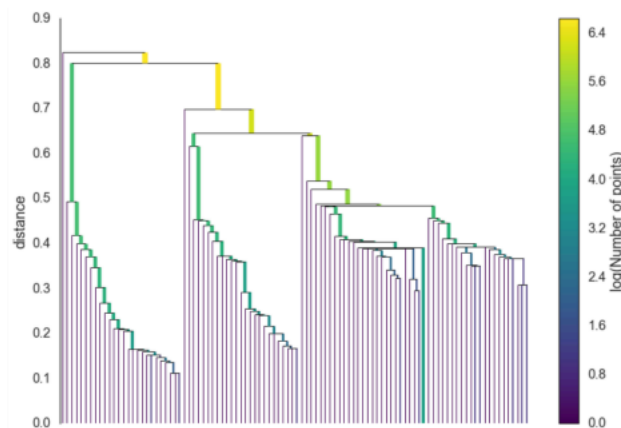


3. Construye un grafo y un árbol jerárquico:

- Vamos a construir un grafo de conectividad sobre la base de la mutual reachability distance.
- Para esto usamos el **minimum spanning tree** usando el algoritmo de Prim. El invariante del **algoritmo de Prim** es agregar la arista de menor peso que conecta al árbol actual un nodo que no está conectado.



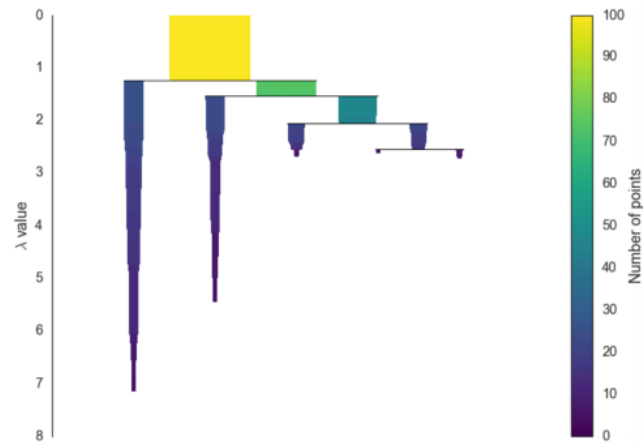
- Dado el árbol de expansión mínima, el siguiente paso es convertirlo en una jerarquía de componentes conectados.
- Esto se hace más fácilmente en orden inverso: se ordenan las aristas del árbol por distancia y luego se itera sobre ellas, creando un nuevo clúster fusionado por cada arista



4. Condensar árbol

Condensaremos bottom-up el dendrograma con una condición de minimum cluster size.

- Si un cluster tiene menos datos, se mezcla con su cluster padre (merge-up).
- Si un split produce dos clusters que cumplen con la condición de tamaño, el split se mantiene



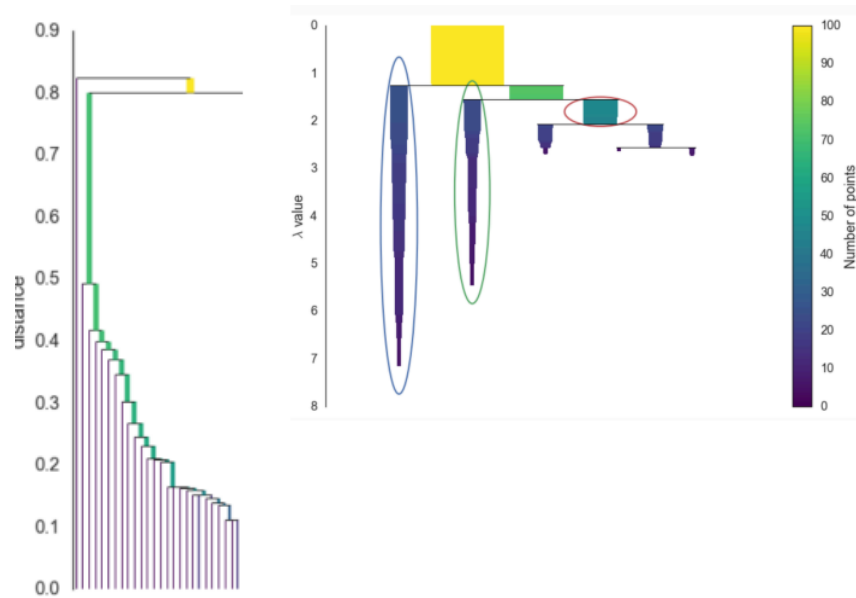
5. Extraer clusters

Se usa el concepto de cluster estable para indicar cuando un cluster sobrevive a los splits definidos por un umbral de distancia en el dendrograma. Es inverso a la distancia, por lo que se define la variable lambda como:

$$\lambda = \frac{1}{\text{distancia}}$$

Hacemos un recorrido top-down del dendrograma. Vamos a tener un lambda de nacimiento del cluster, y en la medida que λ disminuya (cortamos más abajo) en algún momento el cluster va a morir. Para cada dato del cluster vamos a medir cuando sale del cluster (lambda del dato), y calcularemos la estabilidad del cluster según:

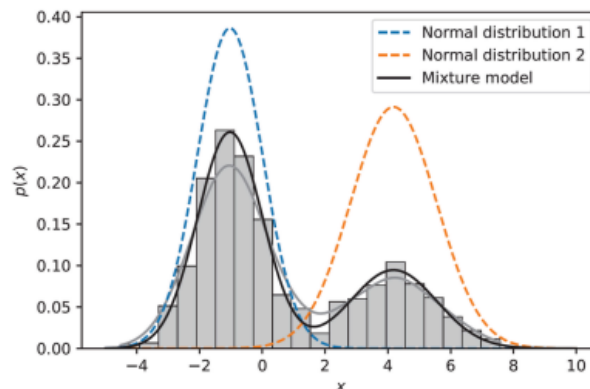
$$\text{Stability}(C) = \sum \frac{1}{\lambda_{\text{death}}(p) - \lambda_{\text{birth}}(p)}$$



Gaussian Mixture Model (GMM)

Es un **modelo probabilístico** que asume que los puntos de datos se generan a partir de una mezcla de varias distribuciones gaussianas con **parámetros desconocidos**.

A diferencia de los métodos de clustering duros como K-Means, que asignan cada punto a un único clúster según el centroide más cercano, el GMM realiza un clustering suave **asignando a cada punto una probabilidad de pertenecer a múltiples clústeres**.



VEROSIMILITUD

La verosimilitud **mide qué tan bien un conjunto de parámetros explica los datos que observamos**. Mide la plausibilidad de diferentes valores de parámetros, dados los datos observados. Es una función de parámetros para datos fijos.

Si tenemos un modelo probabilístico $P(X|\theta)$

$X = \{x_1, x_2, \dots, x_n\}$ son los datos observados

θ son los parámetros del modelo (por ejemplo, media y varianza)

Entonces, la verosimilitud se define como:

$$L(\theta | X) = P(X | \theta) = \prod_{i=1}^n P(x_i | \theta)$$

y su versión logarítmica (más usada) es:

$$\log L(\theta | X) = \sum_{i=1}^n \log P(x_i | \theta)$$

MLE

La estimación de **máxima verosimilitud (MLE)** es un método estadístico que se utiliza para estimar los parámetros de una distribución de probabilidad mediante **maximizar la función de verosimilitud**.

$$\theta = \arg \max L / (\hat{\theta})$$

o bien,

$$\hat{\theta} = \arg \max \sum_{i=1}^n \log P(x_i | \theta)$$

en general se suele utilizar la log-verosimilitud negativa

$$\hat{\theta} = \arg \min \left[- \sum_{i=1}^n \log P(x_i | \theta) \right]$$

Mixture Models

Es un modelo probabilístico que se utiliza para representar **datos que pueden provenir de categorías diferentes, cada una de las cuales se modela mediante una distribución de probabilidad distinta**.

Formalmente, si X es una variable aleatoria cuya distribución es una mezcla de K distribuciones componentes, la función de densidad de probabilidad de X puede escribirse como:

$$p(x) = \sum_{k=1}^K w_k f_k(x; \theta_k)$$

K : es el número de distribuciones componentes en la mezcla.

$f_k()$: es la función de densidad de la distribución componente k -ésima.

w_k : es el peso de mezcla del componente k -ésimo

θ_k : representa los parámetros del componente k -ésimo

GMM

Es un modelo probabilístico que asume que los puntos de datos se generan a partir de una **mezcla de varias distribuciones gaussianas con parámetros desconocidos**.

$$p(x) = \sum_{k=1}^K w_k \mathcal{N}(x; \mu_k, \Sigma_k)$$

K : es el número de distribuciones componentes en la mezcla.

$\mathcal{N}(x; \mu_k, \Sigma_k)$: es la función de densidad normal multivariante para el componente k -ésimo.

w_k : es el peso de mezcla del componente k -ésimo

Σ_k : es la matriz de covarianza del componente gaussiano

μ_k : es el vector de medias del componente gaussiano

GMM

El único parámetro que debemos definir es la cantidad de clusters: K

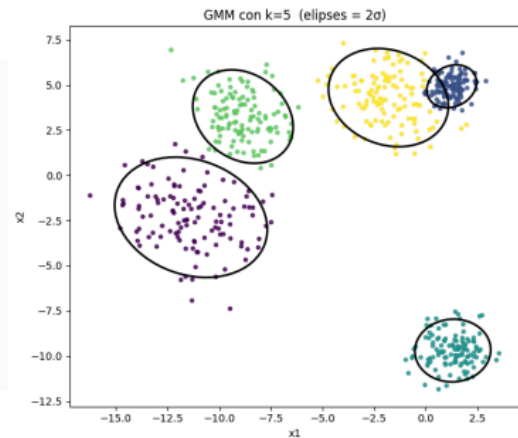
```
from sklearn.mixture import GaussianMixture
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=300, centers=5, cluster_std=[1.3, 0.6, 0.9, 1.9, 1.4])

# Crear y ajustar un modelo GMM con 5 componentes
gmm = GaussianMixture(n_components=5, random_state=42)
gmm.fit(X)

# Predecir a qué componente pertenece cada punto
labels = gmm.predict(X)

plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.title("Clustering con Gaussian Mixture Model (GMM)")
plt.show()
```



Aprender Parámetros GMM

El objetivo es encontrar los parámetros del GMM (medias, covarianzas y coeficientes de mezcla) que mejor expliquen los datos observados

Verosimilitud,

$$\mathcal{L}(\theta | X) = \prod_{i=1}^n \left[\sum_{k=1}^K w_k \mathcal{N}(x_i; \mu_k, \Sigma_k) \right]$$

Log-Verosimilitud,

$$\ell(\theta | X) = \sum_{i=1}^n \log \left[\sum_{k=1}^K w_k \mathcal{N}(x_i; \mu_k, \Sigma_k) \right]$$

No podemos aplicar directamente MLE para estimar los parámetros de un GMM:

- La función de log-verosimilitud es **altamente no lineal** y compleja de maximizar analíticamente
- El modelo **posee variables latentes** (los pesos de mezcla) que **no son directamente observables en los datos**

Expectation-Maximization (EM)

Es un método para **encontrar las estimaciones de máxima verosimilitud** de los parámetros en modelos estadísticos que dependen de variables latentes no observadas, como GMM.

El algoritmo comienza inicializando aleatoriamente los parámetros del modelo, y luego itera entre dos pasos :

1. Expectación (E-step):

Calcula la log-verosimilitud esperada del modelo con respecto a la distribución de las variables latentes, dadas las observaciones y los valores actuales de los parámetros del modelo. En este paso, se **estima la probabilidad de las variables latentes** (la probabilidad de que cada punto pertenezca a cada componente).

2. Maximización (M-step):

Actualiza los parámetros del modelo para maximizar la log-verosimilitud de los datos observados, utilizando las probabilidades de las variables latentes obtenidas en el paso E.

Estos dos pasos se **repiten hasta la convergencia**, la cual suele determinarse mediante un umbral en el cambio de la log-verosimilitud o tras alcanzar un número máximo de iteraciones.

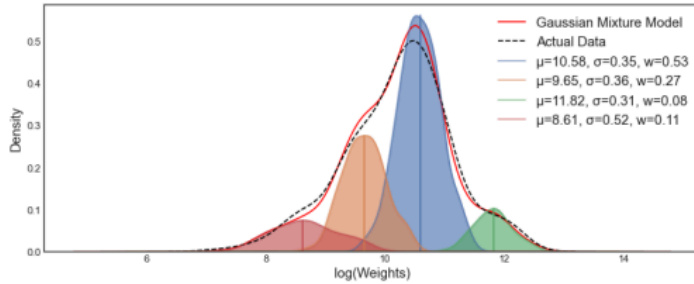
Asignación de Clusters

Una vez que el modelo aprende los parámetros de cada gaussiana, puede calcular la **probabilidad de que un punto x_i haya sido generado por cada componente k** .

Para cada punto x_i , se calcula su responsabilidad o probabilidad posterior de pertenecer al clúster k :

$$\gamma_{ik} = P(k|x_i) = \frac{w_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K w_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}$$

γ_{ik} son las probabilidades de pertenencia, es decir, cuánto contribuye cada gaussiana a explicar el punto



Reglas de Asociación

- Indica que si ocurre el antecedente, es probable que ocurra el consecuente
- Usualmente son variables categóricas nominales.
- Se usan $\{\}$ para indicar asociaciones entre conjuntos denominados itemsets
- La cardinalidad de los conjuntos puede ser mayor a uno: $\{\text{Pan, Mantequilla}\} \rightarrow \{\text{Queso}\}$

ID	Items
1	{Bread, Milk}
2	{Bread, Diapers, Beer, Eggs}
3	{Milk, Diapers, Beer, Cola}
4	{Bread, Milk, Diapers, Beer}
5	{Bread, Milk, Diapers, Cola}



ID	Bread	Milk	Daipers	Beers	Eggs	Cola
1	1	1	0	0	0	0
2	1	0	1	1	1	0
3	0	1	1	1	0	1
4	1	1	1	1	0	0
5	1	1	1	0	0	1

Items

$$I = \{i_1, i_2, i_3, \dots, i_d\}$$

Para la tabla de ejemplo, la cantidad d corresponde a la **cantidad de columnas**

Transacciones

$$T = \{t_1, t_2, t_3, \dots, t_N\}$$

N corresponde a la **cantidad de transacciones o filas**

Soporte de un itemset

$$\sigma(X) = |\{t_i | X \subseteq t_i, t_i \in T\}|$$

Indica cuantas veces aparece una transacción en el conjunto

Support

$$s(X \rightarrow Y) = \frac{\sigma(X \cup Y)}{N}$$

Frecuencia con que aparece la combinación $X \cup Y$

Confidence

$$c(X \rightarrow Y) = \frac{\sigma(X \cup Y)}{\sigma(X)}$$

Probabilidad de comprar Y dado que se compró X

Estrategia para generación de reglas

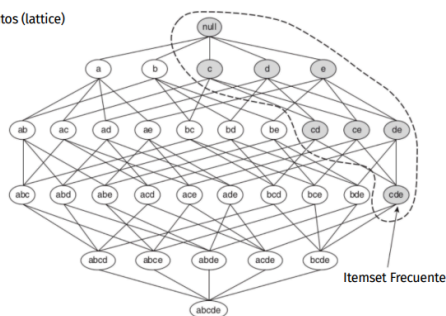
1. Búsqueda de itemsets frecuentes
2. Generación de reglas candidatas a partir de itemsets frecuentes

Buscar asociaciones puede ser muy ineficiente. La eficiencia de la búsqueda es un tema crítico en reglas de asociación

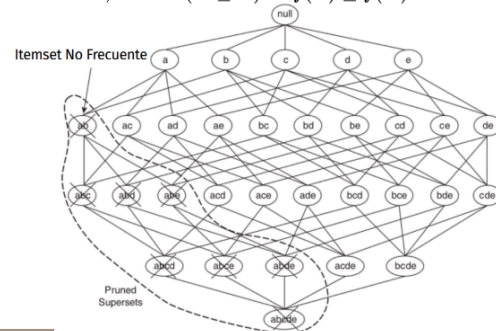
Principio Apriori

Si un itemset es frecuente, entonces todos sus subconjuntos son frecuentes}}

Estructura de subconjuntos (lattice)



Principio de Monotonía $\forall X, Y \in J : (X \subseteq Y) \rightarrow f(Y) \leq f(X)$



Ahora vamos a generar reglas a partir de itemsets frecuentes

Support

$$s(X \rightarrow Y) = \frac{\sigma(X \cup Y)}{N}$$

Confidence

$$c(X \rightarrow Y) = \frac{\sigma(X \cup Y)}{\sigma(X)}$$

Si una regla $X \rightarrow Y - X$ no satisface el umbral de confianza, entonces toda regla de la forma $\tilde{X} \rightarrow Y - \tilde{X}$, donde $\tilde{X} \subseteq X$, no puede satisfacer el umbral de confianza

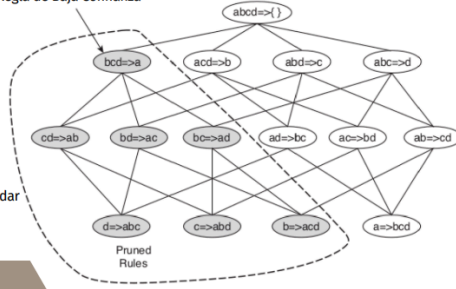
Principio de Monotonicidad $\forall X, Y \in J : (X \subseteq Y) \rightarrow f(Y) \leq f(X)$

$$\tilde{X} \subseteq X \Rightarrow \sigma(X) \leq \sigma(\tilde{X}) \Rightarrow \frac{\sigma(X \cup Y)}{\sigma(X)} \geq \frac{\sigma(\tilde{X} \cup Y)}{\sigma(\tilde{X})}$$

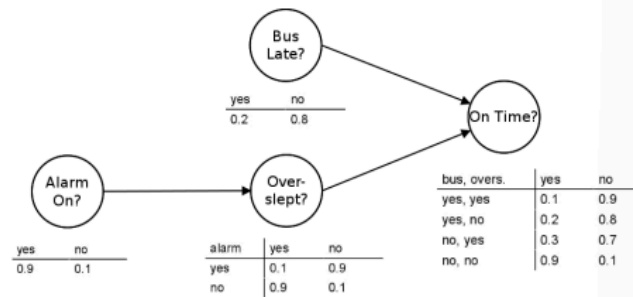
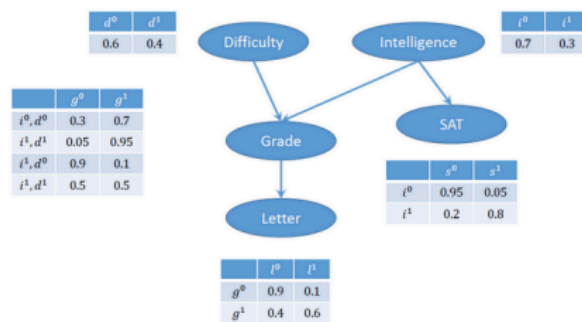
Podemos podar

Si una regla $X \rightarrow Y - X$ no satisface el umbral de confianza, entonces toda regla de la forma $\tilde{X} \rightarrow Y - \tilde{X}$, donde $\tilde{X} \subseteq X$, no puede satisfacer el umbral de confianza

Regla de Baja Confianza



Redes Bayesianas



Fundamentos de probabilidad

Antes de entrar en redes bayesianas, es necesario recordar cómo funcionan las probabilidades simples y condicionales.

- Evento y variable aleatoria
Ej: X = "Tener fiebre", puede ser "sí" o "no".
- Probabilidad conjunta
 $P(A, B) \rightarrow$ probabilidad de que ocurran A y B al mismo tiempo.
- Probabilidad condicional
 $P(A | B) \rightarrow$ probabilidad de A dado que B ocurrió.
 - $P(A|B) = \frac{P(A, B)}{P(B)}$

- Regla del producto

$P(A | B) \rightarrow$ probabilidad de A dado que B ocurrió

- $P(A, B) = P(A|B) \cdot P(B)$

Independencia y Dependencia Condicional

Las redes bayesianas se basan en qué variables dependen o no dependen entre sí.

- Independencia: $P(A, B) = P(A) \cdot P(B)$
 - Dependencia Condicional $P(A|B, C) = P(A|C)$ si A es independiente de B dado C

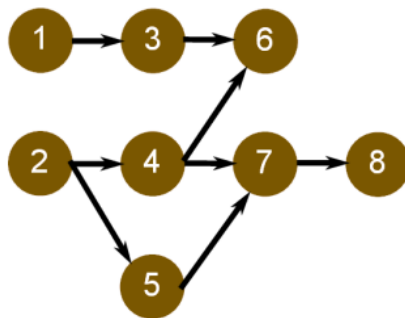
Esto se traduce directamente a la estructura del grafo: si no hay flecha entre dos nodos, puede significar que son condicionalmente independientes

Grafos Dirigidos Acíclicos (DAGs)

Es un tipo de grafo con dos características clave:

Dirigido: Cada arista tiene una dirección específica, apuntando desde un nodo hacia otro.

Acíclico: No contiene "ciclos". Esto significa que es imposible empezar en un nodo, seguir las flechas en su dirección, y terminar de nuevo en el nodo original.

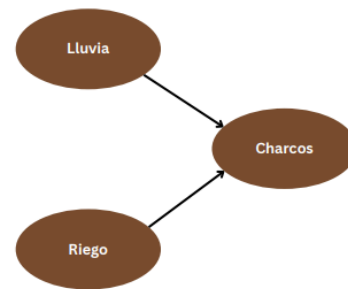


Una red bayesiana se representa con un grafo dirigido sin ciclos:

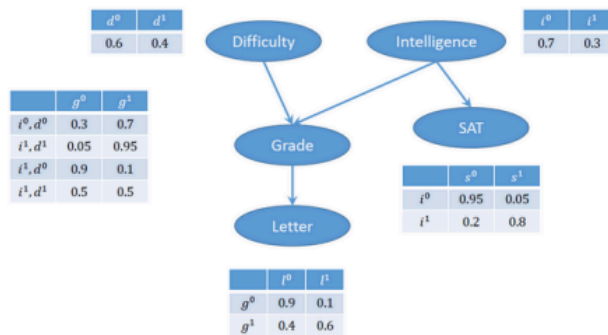
- Nodos $\rightarrow$ variables aleatorias
- Arcos (flechas) $\rightarrow$ dependencias o relaciones causales
- No hay ciclos (no puedes volver al mismo nodo)

Tablas de Probabilidad Condicional (CPT)

Lluvia	Charcos	P(Charcos Lluvia)
Sí	Sí	0.9
Sí	No	0.1
No	Sí	0.2
No	No	0.8



Inferencia Student Bayesian Network



Las tablas no contienen toda la información

Ya hay muchas probabilidades, pero no hay directamente cosas como:

- $P(\text{Intelligence} \mid \text{Grade})$
- $P(\text{Letter} \mid \text{Intelligence})$
- $P(\text{SAT} \mid \text{Letter})$
- $P(\text{Intelligence} \mid \text{SAT}, \text{Letter})$

La inferencia bayesiana significa actualizar creencias cuando existe evidencia parcial.

Ejemplo:

- No se sabe si el curso es difícil ni si el estudiante es inteligente.
- Obtuvo buena nota (Grade = bueno).

¿Cuál es la probabilidad de que sea inteligente dado que obtuvo buena nota?"

Para calcularla, es necesario:

- $P(\text{Intelligence})$
- $P(\text{Difficulty})$
- $P(\text{Grade} \mid \text{Intelligence}, \text{Difficulty})$

$$P(\text{Intelligence} \mid \text{Grade}) = \frac{P(\text{Grade} \mid \text{Intelligence})P(\text{Intelligence})}{P(\text{Grade})}$$

Esto es lo que se conoce a priori

$$P(\text{Intelligence} = \text{Alta}) = 0.3$$

$$P(\text{Difficulty} = \text{Difícil}) = 0.4$$

$$P(\text{Grade} = \text{Buena} \mid \text{Intelligence} = \text{Alta}, \text{Difficulty} = \text{Difícil}) = 0.5$$

Queremos calcular esto

$$P(\text{Intelligence} \mid \text{Grade}) = \frac{P(\text{Grade} \mid \text{Intelligence}) P(\text{Intelligence})}{P(\text{Grade})}$$

Solo necesitamos calcular probabilidades marginal

$$P(\text{Grade} = \text{Buena} \mid \text{Intelligence} = \text{Alta}) = \sum_{d \in \{\text{facil}, \text{difícil}\}} P(\text{Grade} = \text{Buena} \mid \text{Intelligence} = \text{Alta}, \text{Difficulty} = d) P(\text{Difficulty} = d)$$

$$P(\text{Grade} = \text{Buena}) = \sum_{i \in \{\text{baja}, \text{alta}\}} \sum_{d \in \{\text{facil}, \text{difícil}\}} P(\text{Grade} = \text{Buena} \mid i, d) P(i) P(d)$$

d^0	d^1	i^0	i^1
0.6	0.4	0.7	0.3

	d^0	d^1
i^0, d^0	0.3	0.7
i^1, d^1	0.05	0.95
i^1, d^0	0.9	0.1
i^1, d^1	0.5	0.5

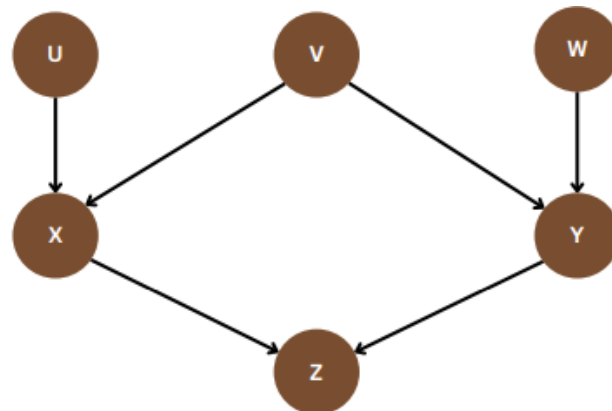
Inferencia: Eliminación de Variables

Es uno de los algoritmos centrales de **inferencia exacta** en Redes Bayesianas. Sirve para calcular **probabilidades marginales o condicionales** cuando la red tiene muchas variables conectadas.

En lugar de sumar sobre todas las combinaciones posibles, V.E. "elimina" variables una a una, combinando solo los factores necesarios en cada paso.

Cada eliminación consiste en:

- Multiplicar los factores que contienen la variable.
- Sumar los valores posibles de esa variable (marginalizarla).
- Guardar el resultado como un nuevo factor reducido.



Query

$$P(U \mid +z)$$

Sin eliminación de variables

$$P(U \mid +z) \propto P(U) \sum_{v,w,x,y} P(v) P(w) P(x \mid U, v) P(y \mid v, w) P(+z \mid x, y)$$

Factores Iniciales

$$P(U); P(V); P(W); P(X \mid U, V); P(Y \mid V, W); P(+z \mid X, Y)$$

Orden de Eliminación

$$W, Y, V, X$$



Inferencia: Variable Elimination

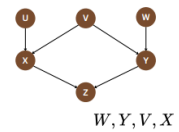
Query $P(U \mid +z)$

Factores

$$P(U); P(V); P(X \mid U, V); P(+z \mid X, Y); f_1(Y, V)$$

$$\text{Eliminar } W \quad f_1(Y, V) = \sum_w P(w) P(Y \mid V, w)$$

$$\text{Eliminar } Y \quad f_2(+z, V, X) = \sum_y f_1(y, V) P(+z \mid X, y)$$



W, Y, V, X

Inferencia: Variable Elimination

Query $P(U \mid +z)$

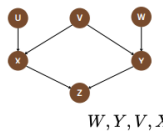
Factores

$$P(U); P(V); P(X \mid U, V); f_2(+z \mid V, X)$$

$$\text{Eliminar } W \quad f_1(Y, V) = \sum_w P(w) P(Y \mid V, w)$$

$$\text{Eliminar } Y \quad f_2(+z, V, X) = \sum_y f_1(y, V) P(+z \mid X, y)$$

$$\text{Eliminar } V \quad f_3(+z, X, U) = \sum_v P(v) P(X \mid U, v) f_2(+z, v, X)$$



W, Y, V, X

Inferencia: Variable Elimination

Query $P(U \mid +z)$

Factores

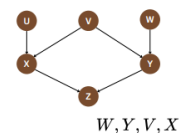
$$P(U); f_3(+z, X, U)$$

$$\text{Eliminar } W \quad f_1(Y, V) = \sum_w P(w) P(Y \mid V, w)$$

$$\text{Eliminar } Y \quad f_2(+z, V, X) = \sum_y f_1(y, V) P(+z \mid X, y)$$

$$\text{Eliminar } V \quad f_3(+z, X, U) = \sum_v P(v) P(X \mid U, v) f_2(+z, v, X)$$

$$\text{Eliminar } X \quad f_4(+z, U) = \sum_x f_3(+z, x, U)$$



W, Y, V, X

Texto en Minería de Datos

Conceptos Importantes

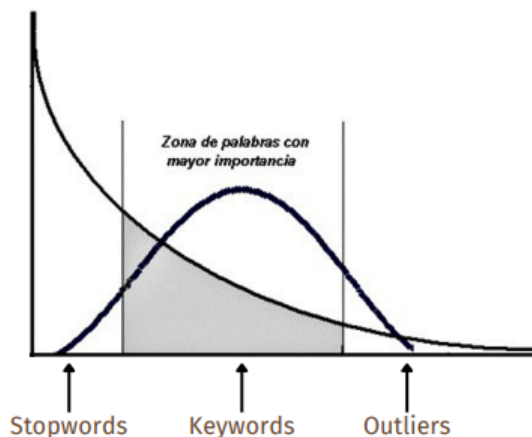
Natural Language Processing

NLP es la rama de la IA que permite a los computadores **entender, generar y razonar** con el lenguaje humano combinando lingüística computacional, aprendizaje estadístico y deep learning.

Stopwords

Las stopwords son palabras muy frecuentes en un idioma (artículos, preposiciones, etc.) que aportan poco significado léxico ("el", "la", "de", "y"). Se suelen eliminar para:

- Reducir el tamaño del vocabulario y el índice.
- Mejorar eficiencia y relevancia al enfocar el análisis en términos con más peso semántico



```
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords

# Stopwords español
spanish_sw = stopwords.words('spanish')
print(spanish_sw)
```

['de', 'la', 'que', 'el', ... , 'tenidas', 'tened']

Conceptos de NLP



Corpus

Colección estructurada de textos usada como datos de entrenamiento o análisis en NLP (p. ej. Brown, Wikipedia, tweets).

0 The Fulton County Grand Jury said Friday an investigation of Atlanta's recent
1 The jury further said in term-end presentments that the City Executive Comm
2 The September-October term jury had been charged by Fulton Superior Cou
3 "Only a relative handful of such reports was received", the jury said, "con
4 The jury said it did find that many of Georgia's registration and election laws
5 It recommended that Fulton legislators act "to have these laws studied and
6 The grand jury commented on a number of other topics, among them the A



Documento

Unidad individual dentro del corpus, como un artículo, un párrafo o un mensaje.

The jury said it did find that many of Georgia's registration and election laws "are outmoded or inadequate and often ambiguous".



Token

Elemento atómico extraído del texto tras la tokenización (palabras, números o símbolos), que sirve como entrada.

['The', 'jury', 'said', 'it', 'did', 'find', 'that', 'many', 'of', 'Georgia', 's', 'registration', 'and', ...]

- Un token no siempre es una palabra, también pueden ser {., ,, ?, ...}

TOKENIZACIÓN

La tokenización es el proceso de segmentar un texto en tokens (por ejemplo palabras, signos de puntuación o subpalabras), que sirven como entrada para algoritmos de NLP

```

from nltk.tokenize import word_tokenize, sent_tokenize, RegexpTokenizer

text = """Coming down like an Armageddon flame (hey!).
The shame, the ones who died without a name"""

# Sentence tokenizer
sentences = sent_tokenize(text)

# Word tokenizer
words = word_tokenize(text)

# Regexp tokenizer
regexp = RegexpTokenizer(r'\w+')
regexp_tokens = regexp.tokenize(text)

Sentence Tokenizer:
['Coming down like an Armageddon flame (hey!).', 'The shame, the ones who died without a name']
Word Tokenizer:
['Coming', 'down', 'like', 'an', 'Armageddon', 'flame', '(', 'hey', '!', ')', 'The', 'shame', ',', 'the', 'ones', 'who', 'died', 'without', 'a', 'name']
Regexp Tokenizer (\w+):
['Coming', 'down', 'like', 'an', 'Armageddon', 'flame', 'hey', 'The', 'shame', 'the', 'ones', 'who', 'died', 'without', 'a', 'name']

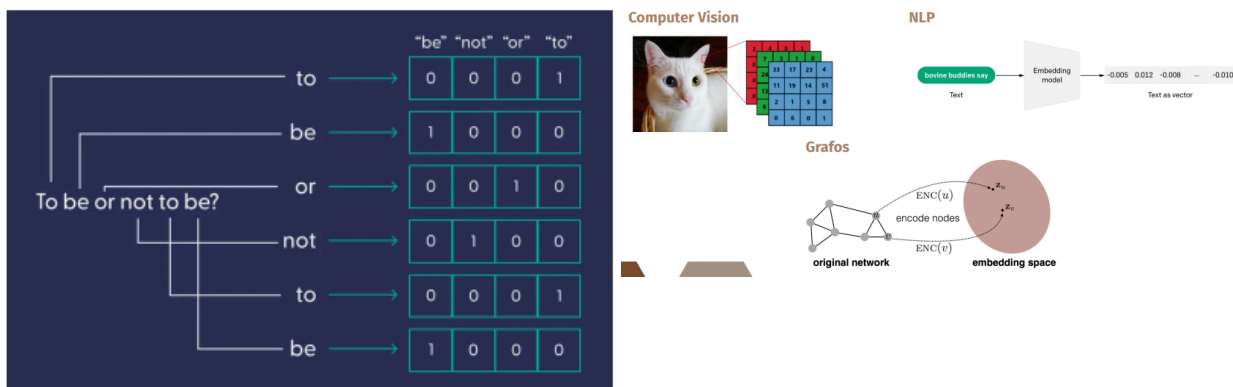
```

N-GRAM

Un n-grama es una secuencia contigua de n unidades (palabras o caracteres) extraídas de un texto. Los n-gramas añaden contexto local que los tokens aislados no capturan (colocaciones, ambigüación).



Representación de Texto



Bag of Words (BoW)

Es una de las formas mas basicas de representar texto en NLP e IR. Consiste en:

1. Tomar un texto (frase, párrafo, documento).
2. Dividirlo en sus palabras (tokens).
3. Contar cuantas veces aparece cada palabra.
4. Guardar esas cuentas en un vector.

	about	bird	heard	is	the	word	you
About the bird, the bird, bird bird bird	1	5	0	0	2	0	0
You heard about the bird	1	1	1	0	1	0	1
The bird is the word	0	1	0	1	2	1	0

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "El perro corre rápido y el perro ladra fuerte.",
    "La casa es grande, la casa es bonita.",
    "Me gusta la música, la música es mi pasión.",
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)

print("Vocabulario:", vectorizer.get_feature_names_out())
print(X.toarray())

Vocabulario: ['bonita' 'casa' 'corre' 'el' 'es' 'fuerte' 'grande' 'gusta' 'la' 'ladra'
              'me' 'mi' 'música' 'pasión' 'perro' 'rápido']
[[0 0 1 2 0 1 0 0 0 0 1 0 0 0 0 2 1]
 [1 2 0 0 2 0 1 0 2 0 0 0 0 0 0 0 0]
 [0 0 0 0 1 0 0 1 2 0 1 1 2 1 0 0 0]]
```

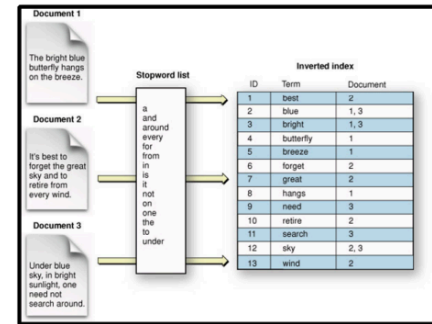
TF-IDF

Es una mejora sobre Bag of Words porque no solo cuenta cuántas veces aparece una palabra en un documento, sino también **qué tan importante es esa palabra en todo el corpus**.

- En los 70s el área de IR estaba en auge por la necesidad de organizar grandes colecciones de documentos
- Se usaban conteos brutos (BoW) que no distinguían palabras comunes de palabras clave.
- En este contexto, varios investigadores (notablemente Karen Spärck Jones en 1972) introdujeron la idea de la frecuencia inversa de documentos (IDF).



Índice Invertido



Cuántas veces aparece una palabra en un texto, comparado con la longitud total de ese texto.

$$TF(t, d) = \frac{\text{Cantidad de veces que } t \text{ aparece en } d}{\text{Total de términos en } d}$$

Le da más valor a las palabras que aparecen en pocos documentos de todo el conjunto, y menos a las que salen en casi todos

$$IDF(t) = \log \left(\frac{\text{Cantidad total de documentos}}{\text{Cantidad de documentos que contienen el término } t} \right)$$

Combina ambos para destacar las palabras que salen con frecuencia en un texto pero son raras en el resto del corpus.

$$TD\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

```
from nltk.corpus import stopwords
spanish_stopwords = stopwords.words("spanish")
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "La inflación sube en Chile: inflación, inflación e IPC empujan precios; ",
    "el IPC mensual refleja inflación en alimentos y energía.",
    "Bancos centrales monitorean inflación e IPC; inflación moderada, proyecciones de ",
    "crecimiento y crecimiento global.",
    "Chile gana el clásico: goles, goles y más goles; la selección de Chile presiona alto ",
    "y mantiene posesión.",
    "Autos eléctricos, autos eléctricos y más autos eléctricos: mejora de batería, batería ",
    "y autonomía en nuevos modelos.",
    "El Congreso de Chile discute reforma, reforma constitucional; la reforma busca ",
    "descentralización y consenso.",
    "Santiago incorpora autos eléctricos al transporte público: recarga, recarga rápida y ",
    "puntos de recarga para autos."
]

# Vectorizador TF-IDF
tfidf = TfidfVectorizer(stop_words=spanish_stopwords, lowercase=True, max_features=30,
                        ngram_range=(1,1))

X = tfidf.fit_transform(corpus)
vocab = np.array(tfidf.get_feature_names_out())
X_dense = X.toarray()
```

Para qué sirven los vectores de texto?

Los vectores son la puerta de entrada para aplicar algoritmos de Machine Learning.

1. Clasificación de Texto
 - a. Spam vs No Spam
 - b. Sentiment Analysis
 - c. Clasificación de Noticias
2. Clustering
 - a. Encontrar grupos de documentos similares
3. Búsqueda e IR
 - a. Comparar consultas vs documentos usando similaridad de coseno.

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

# Corpus y etiquetas (1=positivo, 0=negativo)
corpus = [
    "Me encanta este producto de gran calidad",
    "El servicio fue excelente y muy rápido",
    "Estoy muy satisfecho con mi compra",
    "La comida estaba deliciosa y bien servida",
    "Recomiendo totalmente esta tienda a mis amigos",
    "Odio este producto tan malo y defectuoso",
    "El servicio fue terrible y muy lento",
    "Estoy muy decepcionado con la experiencia",
    "La comida estaba fría y mal servida",
    "No recomiendo esta tienda por su atención"
]
labels = [1,1,1,1,1, 0,0,0,0,0]

# Vectorizar
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)

# Modelo
clf = LogisticRegression()
clf.fit(X, labels)

# Predicción de ejemplo
texto = ["El producto llegó rápido y funciona bien"]
print("Predicción:", clf.predict(vectorizer.transform(texto)))
print("Probabilidades:", clf.predict_proba(vectorizer.transform(texto)))

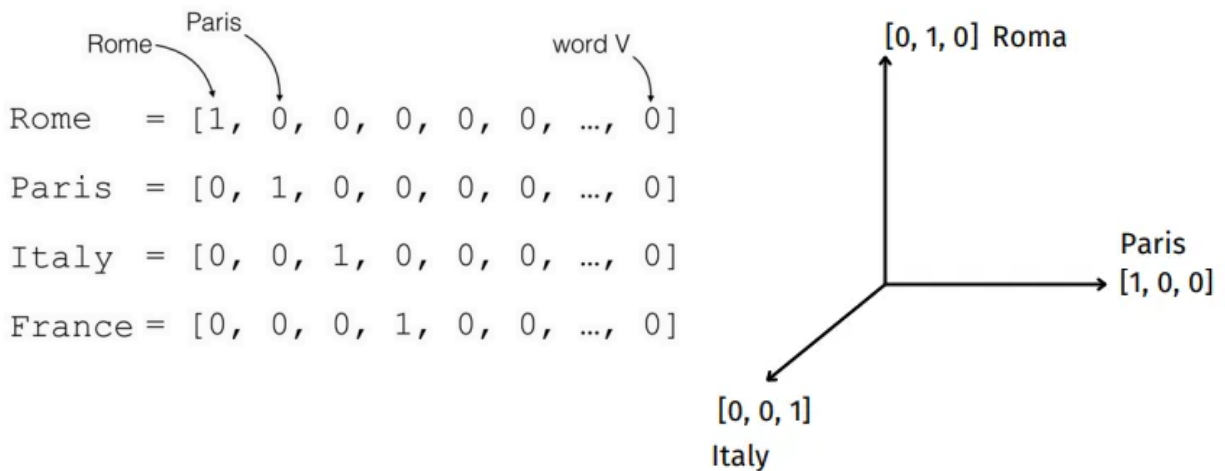
Predicción: [1]
Probabilidades: [[0.44268013 0.55731987]]

```

Modelos de Lenguaje

Vector-Space Model

Cada término se representa como una dimensión del vector



La distancia entre cada par de vector siempre es la misma → cada palabra vale por igual, aunque semánticamente no se parecen

Cuando tenemos una situación de **polisemia** los vectores se representan de la misma forma sin hacer distinción del contexto

Tengo que ir a pagar al **banco**

Deberían arreglar el **banco** de la plaza

banco en Doc 1: [0, 0, 1, 0, 0]
banco en Doc 2: [0, 0, 1, 0, 0]

Limitaciones

Pensemos en que tenemos un vocabulario de más de 30.000 palabras. Cada palabra está representado por un vector de 30.000 dimensiones (V)

I: [0, 0, 0, 0, 0, 1, 0, ..., ..., 0, 0]
cat: [0, 0, 0, 1, 0, 0, 0, ..., ..., 0, 0]
dog: [0, 1, 0, 0, 0, 0, 0, ..., ..., 0, 0]
have: [0, 0, 0, 0, 0, 0, 0, ..., ..., 1, 0]
a: [0, 0, 0, 0, 0, 0, 0, ..., ..., 0, 1]
} 30.000 dim

- Muy disperso (alta dimensionalidad).
- No captura relaciones semánticas (no sabe que gato y perro son parecidos).

Word2Vec

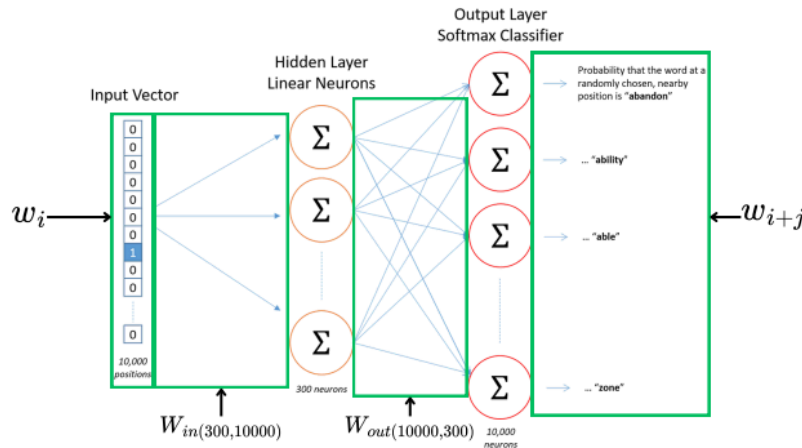
Word2Vec es un modelo creado por Tomas Mikolov y su equipo en Google (2013) que aprende a **representar palabras como vectores numéricos densos** que capturan el significado

Palabra	Vector (dim=4)
gato	[0.21, -0.34, 0.11, 0.78]
perro	[0.25, -0.31, 0.09, 0.74]
mesa	[-0.51, 0.22, -0.33, 0.05]

Cómo funciona

Word2Vec es una red neuronal muy pequeña que aprende de texto sin etiquetas

Variante	Qué predice	Ejemplo
CBOW	Predice la palabra central a partir de las palabras del contexto.	Contexto: "El __ corre" → predice "perro"
Skip-Gram	Predice el contexto a partir de la palabra central.	"perro" → predice "El", "corre"



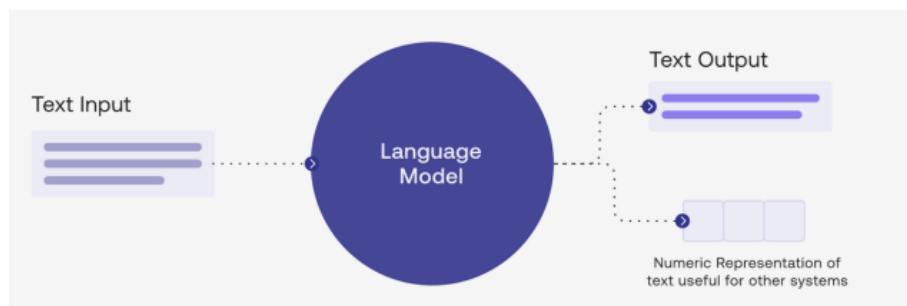
Limitaciones

- **Embeddings estáticos (sin contexto):** la palabra tiene un solo vector sin importar la frase. "banco" (parque vs. entidad financiera) queda mezclado.
- **Out-of-Vocabulary (OOV) y morfología:** no modela prefijos/sufijos ni caracteres. Palabras nuevas o con faltas $\rightarrow$ UNK. En lenguas ricas en morfología (español), pierde información. "barato" $\rightarrow$ "baratísimo" $\rightarrow$ "baratito"
- **Vocabulario cerrado y corte por frecuencia:** hay que fijar un vocabulario; las palabras bajo el umbral de frecuencia quedan fuera o con vectores pobres

Modelos de lenguaje

Un modelo de lenguaje es un sistema que aprende a predecir la probabilidad de una secuencia de palabras en un idioma.

- Representan texto como vectores semánticos en espacios de alta dimensión.
- Capturan orden, contexto y significado

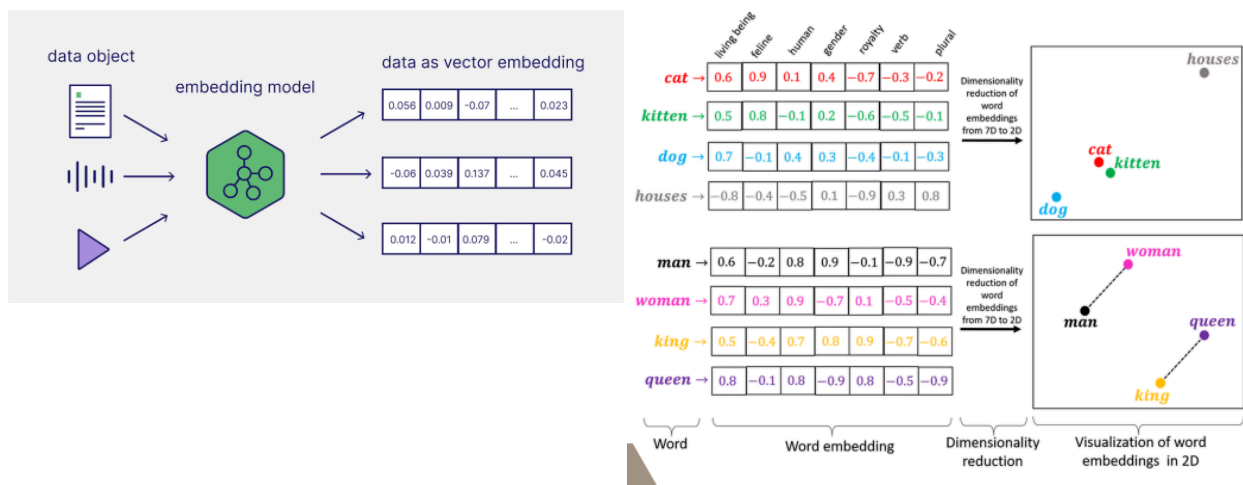


Un modelo de lenguaje es cualquier sistema que predice o genera texto en base a probabilidades de secuencias de palabras. Un **Large Language Model (LLM)** es un tipo de modelo de lenguaje mucho más grande y poderoso, entrenado con miles de millones de parámetros y enormes cantidades de datos, lo que le permite no solo predecir palabras, sino también generar texto coherente, responder preguntas, razonar y adaptarse a múltiples tareas

Metric	LLaMA 3.1	GPT-4	BERT	T5
Parameter Count	300 billion	175 billion	340 million	11 billion
Training Data Size	2.5 TB of text data	1.5 TB of text data	3.3 billion words	750 GB of text data
GLUE Score	92.5	90.5	80.5	89.9
SQuAD Score	90.3	88.9	86.7	87.7
SuperGLUE Score	89.7	87.4	78.3	86.2

Embedding de Texto

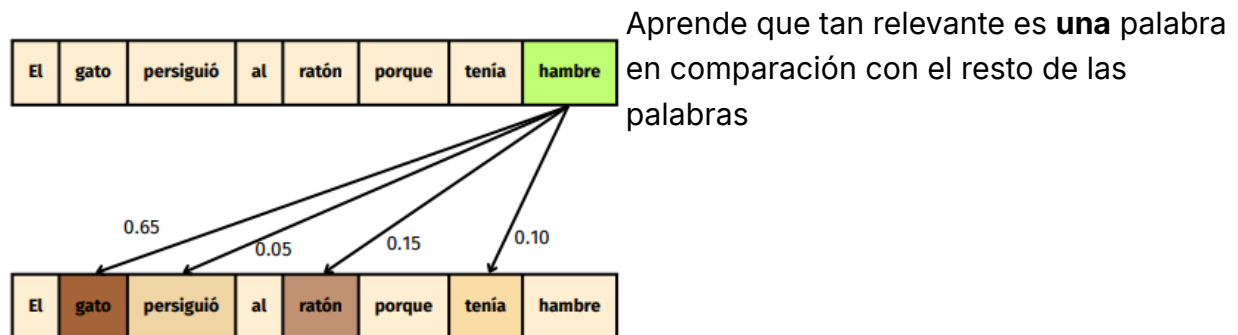
Un embedding es una representación numérica de un objeto (palabra, frase, documento, imagen, etc.) en forma de vector dentro de un espacio matemático



Atención

Attention Is All You Need (Vaswani, 2017)

Marcó un antes y un después en la historia de la inteligencia artificial. Su impacto fue tan grande que transformó completamente el campo de NLP y, con el tiempo, toda la IA moderna.



Es el **mecanismo de autoatención (self-attention)** permite que cada palabra aprenda su relación con todas las demás sin depender de un orden secuencial

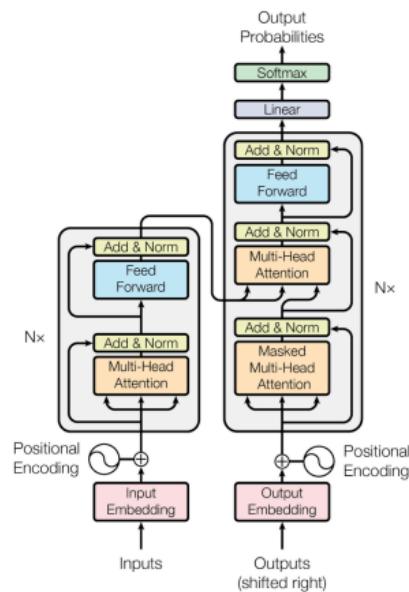
- Ya no hay hidden state que se pasa paso a paso.
- El modelo ve todo el contexto a la vez.
- Esto permitió paralelización masiva, dependencias largas y mayor estabilidad de entrenamiento.



Transformer

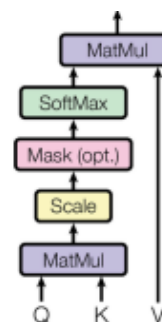
En lugar de procesar la información palabra por palabra (como lo hacían las RNN o LSTM), el Transformer **procesa toda la secuencia a la vez** y aprende qué partes del texto deben prestarse más atención unas a otras.

- Encoder: lee la secuencia de entrada completa . Calcula representaciones internas (embeddings contextuales) para cada palabra, entendiendo su relación con las demás.
- Decoder: genera una salida paso a paso (por ejemplo, una traducción). Usa la información del encoder + atención hacia palabras ya generadas



- Self-Attention: cada palabra genera tres vectores: Q (Query), K (Key) y V (Value). El modelo calcula qué tan relevante es cada palabra respecto a las demás usando el producto QK. Los pesos resultantes se usan para combinar los valores V

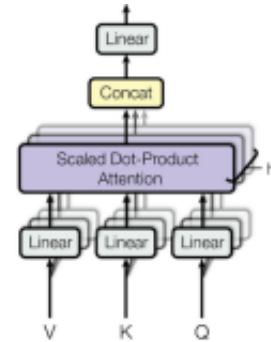
$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$



- Multi-Head Attention: en lugar de una sola atención, el modelo usa varios "cabezales" que observan la secuencia desde diferentes perspectivas. Luego concatena sus resultados para formar una representación más rica del contexto.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$



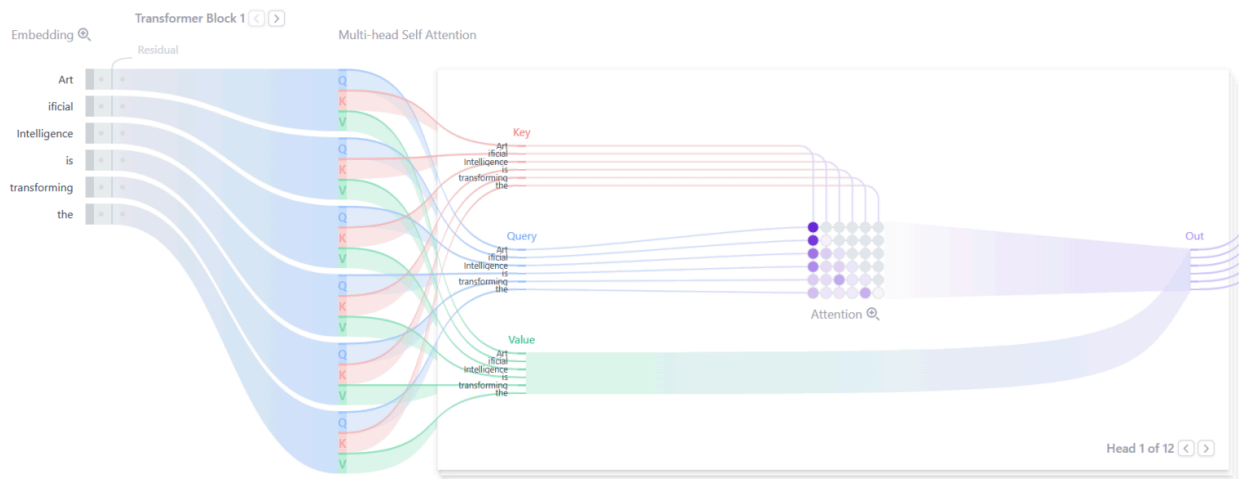
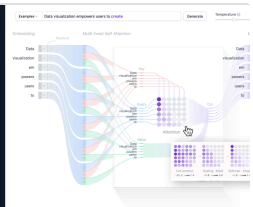
Transformer Explainer: LLM Transformer Model Visually Explained

An interactive visualization tool showing you how transformer models work in large language models (LLM) like GPT.

<https://poloclub.github.io/transformer-explainer/>

TRANSFORMER EXPLAINER

Learn how transformer works in large language models like GPT!



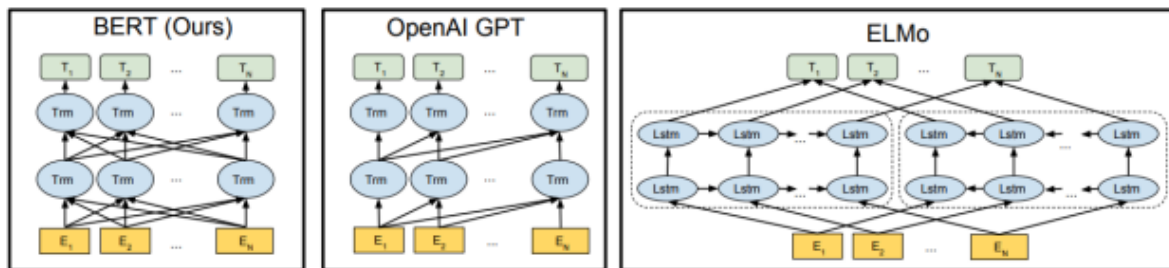
Bert

BERT (Bidirectional Encoder Representations from Transformers) fue propuesto por Devlin et al. (Google, 2018) y cambió por completo la forma de entrenar modelos de lenguaje.

Aprende embeddings del lenguaje usando solo la parte encoder del Transformer.

A diferencia de modelos anteriores (como GPT-1 o ELMo), BERT:

- Lee el texto **en ambas direcciones** (izquierda y derecha) **al mismo tiempo** → "bidireccional".
- Captura relaciones de contexto mucho más robustas



CÓMO SE ENTRENA?

BERT no se entrena directamente para una tarea específica, sino con tareas de preentrenamiento:

1. Masked Language Modeling (MLM)

- a. Se ocultan algunas palabras del texto (por ejemplo, 15%)
- b. El modelo debe predecir las palabras faltantes.
- c. Ejemplo:
 - i. "El [MASK] persiguió al ratón" → predice "gato".

2. Next Sentence Prediction (NSP)

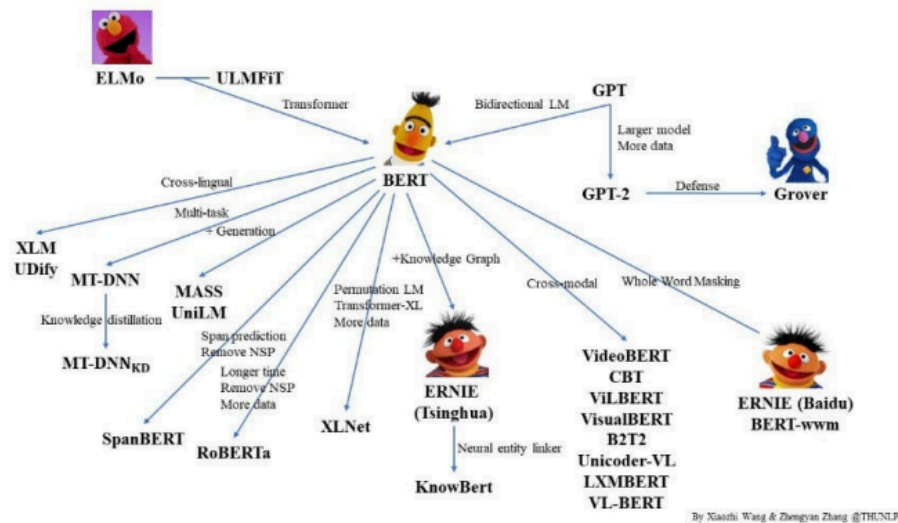
- a. Se le dan dos frases y debe predecir si la segunda sigue a la primera.
- b. Ayuda a aprender relaciones entre oraciones.

Luego, el **modelo puede ajustarse (fine-tuning) para tareas específicas**: clasificación de texto, análisis de sentimientos, NER, QA, etc.

Modelos Pre-entrenados

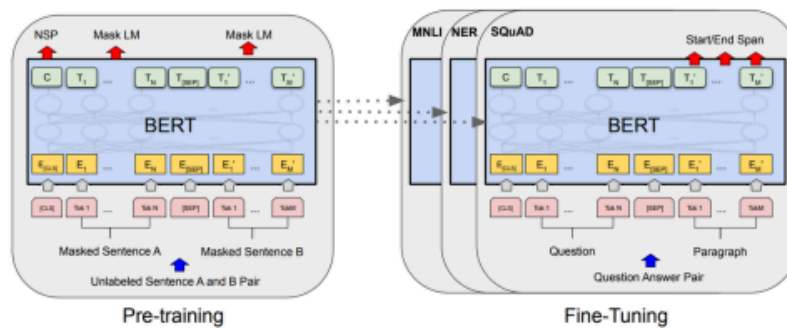
Un modelo pre-entrenado es un modelo de machine learning o deep learning que ya fue entrenado previamente en una gran cantidad de datos y luego se pone a

disposición para que otras personas lo usen o lo ajusten (fine-tuning) en tareas específicas



Fine-Tuning

El fine-tuning es el proceso de tomar un modelo ya pre-entrenado (como BERT o GPT) y ajustarlo con un conjunto de datos específico para una tarea concreta.



System	MNLI-(m/mm)	QQP	QNLI	SST-2	CoLA	STS-B	MRPC	RTE	Average
	392k	363k	108k	67k	8.5k	5.7k	3.5k	2.5k	-
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

